

DATABASE DESIGN ASSIGNMENT · P1 & P2

BookMyShow-Scale Ticketing Backend

Normalised MySQL schema, seat-level locking with timed holds,
idempotent payment webhooks, and a concurrency load test that proves it

MySQL 8.0 · InnoDB

BCNF

Pessimistic row locks

Timed holds (lazy expiry)

Idempotent webhooks

500-user load test

Submitted by **Sai**

Deliverables This document · sql/01-05 · load_test/load_test.py

Verified on MySQL 8.0.46 — every example row and result in this document was produced by running the submitted SQL

Contents

1	Scenario and scope
2	Entities and ER diagram
3	Tables, attributes and example rows
4	Normalisation: 1NF, 2NF, 3NF, BCNF
5	Concurrency and locking strategy
6	Pessimistic vs optimistic locking
7	Indexing and query optimisation
8	P2 – shows at a theatre on a date
9	Load and concurrency test
10	How to run
A	Appendix A – P1 SQL: schema (DDL)
B	Appendix B – P1 SQL: locking procedures
C	Appendix C – P1 SQL: sample data
D	Appendix D – P2 SQL
E	Appendix E – Walk-through output

1. Scenario and scope

BookMyShow sells cinema tickets. For a chosen theatre the app shows the next 7 dates; picking a date lists every movie playing there that day with its show timings; picking a timing opens the seat map where the user selects seats, pays, and receives a ticket. At scale, **thousands of users compete for the same seats** in the same second (first-day-first-show of a blockbuster), so the backend must guarantee:

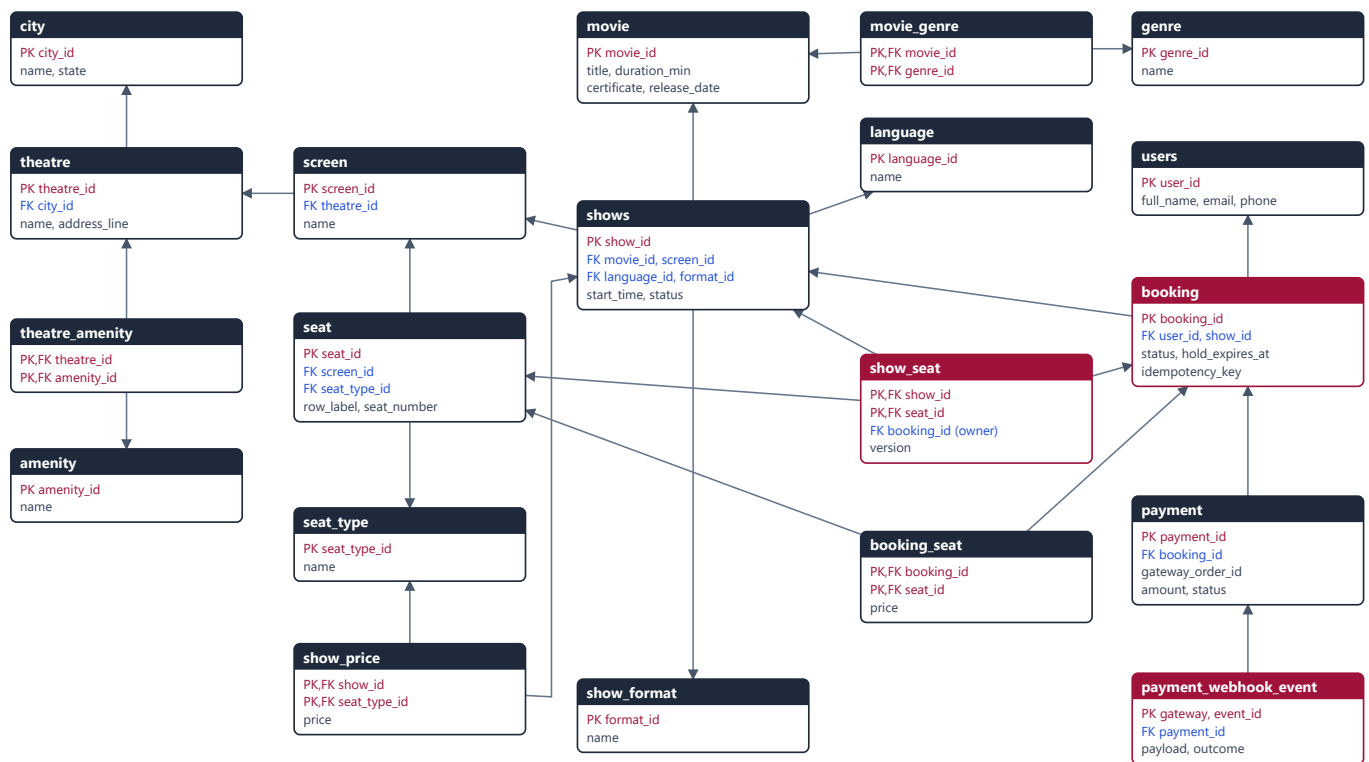
1. **No double booking** — a seat of a show is sold to at most one customer, under any interleaving of requests.
2. **No lost holds** — while a customer is paying, their held seats cannot be taken; and a hold that is abandoned is released automatically on a timer, so inventory is never stuck.
3. **Idempotent payments** — payment gateways deliver webhooks *at least once* (retries, duplicates, out of order, late). Each payment must change state exactly once.
4. **Fast reads** — the theatre → date → shows listing (P2) is the most frequent query and must use indexes.

What is delivered. P1: 20 tables in BCNF with DDL, sample data, and stored procedures implementing the locking strategy (sql/01_schema.sql, 02_procedures.sql, 03_sample_data.sql). **P2:** the listing query in flat and UI-grouped form plus the 7-day date strip (sql/04_...sql). A walk-through with integrity checks (sql/05_...sql) and a load test (load_test/) prove the guarantees. All scripts are plain MySQL and run unmodified in the mysql client or MySQL Workbench.

2. Entities and ER diagram

Group	Entities	Role
Venue	city, theatre, amenity, theatre_amenity, screen, seat_type, seat	Physical world: where, which auditorium, which chair.
Catalogue	movie, genre, movie_genre, language, show_format	What is being shown.
Scheduling	shows, show_price	A movie on a screen at a time, in a language/format, and its prices.
Selling	users, booking, booking_seat, show_seat	Holds, bookings and the per-seat inventory that enforces one owner.
Money	payment, payment_webhook_event	Gateway orders and the de-duplicated webhook log.

Relationships (all 1 : N unless noted): city $\rightarrow$ theatre $\rightarrow$ screen $\rightarrow$ seat; theatre $\leftrightarrow$ amenity (M : N); movie $\leftrightarrow$ genre (M : N); movie, screen, language, format $\rightarrow$ shows; shows $\times$ seat_type $\rightarrow$ show_price; shows $\times$ seat $\rightarrow$ show_seat; users, shows $\rightarrow$ booking; booking $\times$ seat $\rightarrow$ booking_seat; booking $\rightarrow$ payment $\rightarrow$ payment_webhook_event.



Arrows point from the child (foreign key) to the parent (primary key). Red headers mark the three tables at the heart of the concurrency design.

3. Tables, attributes and example rows

For every table: its purpose, attributes (type, nullability, keys), its functional dependencies and candidate keys (used in §4 to prove BCNF), and example rows taken from the sample database. Show dates in the sample data are relative to the day the script is run (today ... today + 6), exactly like the app's 7-day strip, so the P2 query always returns data.

3.1 city 2 rows in sample DB

A city in which theatres operate.

Attribute	Type	Null	Keys	Default / extra
city_id	smallint unsigned	NOT NULL	PK	auto_increment
name	varchar(80)	NOT NULL	UQ	
state	varchar(80)	NOT NULL	UQ	

Functional dependencies: city_id → name, state ; (name, state) → city_id

Candidate keys: {city_id}, {name, state}

Example rows

city_id	name	state
2	Bengaluru	Karnataka
1	Hyderabad	Telangana

3.2 theatre 3 rows in sample DB

A cinema (venue). Belongs to exactly one city.

Attribute	Type	Null	Keys	Default / extra
theatre_id	int unsigned	NOT NULL	PK	auto_increment
city_id	smallint unsigned	NOT NULL	FK → city.city_id UQ	
name	varchar(120)	NOT NULL	UQ	
address_line	varchar(255)	NOT NULL		
created_at	datetime	NOT NULL		default CURRENT_TIMESTAMP

Functional dependencies: theatre_id → city_id, name, address_line, created_at ; (city_id, name) → theatre_id

Candidate keys: {theatre_id}, {city_id, name}

Example rows

theatre_id	city_id	name	address_line
1	1	Aurora Cinemas: Kukatpally	4th Floor, Metro Square Mall, Kukatpally, Hyderabad 500072
2	1	Starlight Multiplex: Gachibowli	Level 3, Orbit Towers, Gachibowli, Hyderabad 500032
3	2	Galaxy Screens: Koramangala	80 Feet Road, 4th Block, Koramangala, Bengaluru 560034

3.3 amenity 4 rows in sample DB

Facility a theatre can offer (M-Ticket, Food & Beverage ...).

Attribute	Type	Null	Keys	Default / extra
amenity_id	smallint unsigned	NOT NULL	PK	auto_increment
name	varchar(60)	NOT NULL	UQ	

Functional dependencies: amenity_id ↔ name

Candidate keys: {amenity_id}, {name}

Example rows

amenity_id	name
2	Food & Beverage
1	M-Ticket
3	Parking
4	Wheelchair Access

3.4 theatre_amenity 9 rows in sample DB

Junction table: which theatre offers which amenity (M:N).

Attribute	Type	Null	Keys	Default / extra
theatre_id	int unsigned	NOT NULL	PK FK → theatre.theatre_id	
amenity_id	smallint unsigned	NOT NULL	PK FK → amenity.amenity_id	

Functional dependencies: no non-trivial FDs (all-key relation)

Candidate keys: {theatre_id, amenity_id}

Example rows

theatre_id	amenity_id
1	1
1	2
1	3
1	4
2	1

3.5 screen 5 rows in sample DB

An auditorium inside a theatre.

Attribute	Type	Null	Keys	Default / extra
screen_id	int unsigned	NOT NULL	PK	auto_increment
theatre_id	int unsigned	NOT NULL	FK → theatre.theatre_id UQ	
name	varchar(40)	NOT NULL	UQ	

Functional dependencies: screen_id → theatre_id, name ; (theatre_id, name) → screen_id

Candidate keys: {screen_id}, {theatre_id, name}

Example rows

screen_id	theatre_id	name
1	1	Audi 1
2	1	Audi 2
3	1	IMAX
4	2	Screen 1
5	3	Screen 1

3.6 seat_type 3 rows in sample DB

Seat category that drives pricing (CLASSIC, PRIME, RECLINER).

Attribute	Type	Null	Keys	Default / extra
seat_type_id	tinyint unsigned	NOT NULL	PK	auto_increment
name	varchar(30)	NOT NULL	UQ	

Functional dependencies: seat_type_id ↔ name

Candidate keys: {seat_type_id}, {name}

Example rows

seat_type_id	name
1	CLASSIC
2	PRIME
3	RECLINER

3.7 seat 400 rows in sample DB

A physical seat in a screen, identified by row + number.

Attribute	Type	Null	Keys	Default / extra
seat_id	bigint unsigned	NOT NULL	PK	auto_increment
screen_id	int unsigned	NOT NULL	FK → screen.screen_id UQ	
row_label	varchar(2)	NOT NULL	UQ	
seat_number	tinyint unsigned	NOT NULL	UQ	
seat_type_id	tinyint unsigned	NOT NULL	FK → seat_type.seat_type_id	

Functional dependencies: seat_id → screen_id, row_label, seat_number, seat_type_id ; (screen_id, row_label, seat_number) → seat_id

Candidate keys: {seat_id}, {screen_id, row_label, seat_number}

Example rows

seat_id	screen_id	row_label	seat_number	seat_type_id
1	1	A	1	1
2	1	A	2	1
41	1	E	1	2
42	1	E	2	2

seat_id	screen_id	row_label	seat_number	seat_type_id
71	1	H	1	3
72	1	H	2	3

3.8 movie 5 rows in sample DB

A film in the catalogue.

Attribute	Type	Null	Keys	Default / extra
movie_id	int unsigned	NOT NULL	PK	auto_increment
title	varchar(150)	NOT NULL	UQ	
duration_min	smallint unsigned	NOT NULL		
certificate	enum('U','UA','A','S')	NOT NULL		
release_date	date	NOT NULL	UQ	

Functional dependencies: movie_id → title, duration_min, certificate, release_date ; (title, release_date) → movie_id

Candidate keys: {movie_id}, {title, release_date}

Example rows

movie_id	title	duration_min	certificate	release_date
1	Monsoon Heist	152	UA	2026-09-18
2	Iron Valley	169	UA	2026-09-25
3	Chasing Tides	134	U	2026-09-11
4	Little Astronaut	108	U	2026-08-28
5	The Midnight Ledger	141	A	2026-09-25

3.9 genre 7 rows in sample DB

A genre label.

Attribute	Type	Null	Keys	Default / extra
genre_id	tinyint unsigned	NOT NULL	PK	auto_increment
name	varchar(40)	NOT NULL	UQ	

Functional dependencies: genre_id ↔ name

Candidate keys: {genre_id}, {name}

Example rows

genre_id	name
1	Action
5	Animation
7	Crime
3	Drama
4	Family

3.10 movie_genre 12 rows in sample DB

Junction table: a movie has many genres (M:N) – keeps genres atomic (1NF).

Attribute	Type	Null	Keys	Default / extra
movie_id	int unsigned	NOT NULL	PK FK → movie.movie_id	
genre_id	tinyint unsigned	NOT NULL	PK FK → genre.genre_id	

Functional dependencies: no non-trivial FDs (all-key relation)

Candidate keys: {movie_id, genre_id}

Example rows

movie_id	genre_id
1	1
2	1
1	2
5	2
2	3

3.11 language 5 rows in sample DB

Audio language of a show.

Attribute	Type	Null	Keys	Default / extra
language_id	tinyint unsigned	NOT NULL	PK	auto_increment
name	varchar(40)	NOT NULL	UQ	

Functional dependencies: language_id ↔ name

Candidate keys: {language_id}, {name}

Example rows

language_id	name
1	English
2	Hindi
5	Kannada
4	Tamil
3	Telugu

3.12 show_format 4 rows in sample DB

Projection format of a show (2D, 3D, IMAX 2D, 4DX).

Attribute	Type	Null	Keys	Default / extra
format_id	tinyint unsigned	NOT NULL	PK	auto_increment
name	varchar(30)	NOT NULL	UQ	

Functional dependencies: format_id ↔ name

Candidate keys: {format_id}, {name}

Example rows

format_id	name
1	2D
2	3D
4	4DX
3	IMAX 2D

3.13 shows 147 rows in sample DB

One screening: a movie, on a screen, in a language and format, at a start time. (Named shows because SHOW is a reserved word in MySQL.)

Attribute	Type	Null	Keys	Default / extra
show_id	bigint unsigned	NOT NULL	PK	auto_increment
movie_id	int unsigned	NOT NULL	FK → movie.movie_id	
screen_id	int unsigned	NOT NULL	FK → screen.screen_id UQ	
language_id	tinyint unsigned	NOT NULL	FK → language.language_id	
format_id	tinyint unsigned	NOT NULL	FK → show_format.format_id	
start_time	datetime	NOT NULL	UQ	
status	enum('SCHEDULED', 'CANCELLED')	NOT NULL		default SCHEDULED
cancellation_allowed	tinyint(1)	NOT NULL		default 1

Functional dependencies: show_id → movie_id, screen_id, language_id, format_id, start_time, status, cancellation_allowed ;
(screen_id, start_time) → show_id

Candidate keys: {show_id}, {screen_id, start_time}

Example rows

show_id	movie_id	screen_id	language_id	format_id	start_time	status	cancellation_allowed
22	2	1	3	1	2026-09-29 10:00:00	SCHEDULED	1
23	2	1	3	1	2026-09-29 13:45:00	SCHEDULED	1
24	1	1	2	1	2026-09-29 17:30:00	SCHEDULED	1
25	2	1	3	1	2026-09-29 21:15:00	SCHEDULED	1
31	2	3	3	3	2026-09-29 11:00:00	SCHEDULED	1

3.14 show_price 441 rows in sample DB

Ticket price of a seat type for a particular show.

Attribute	Type	Null	Keys	Default / extra
show_id	bigint unsigned	NOT NULL	PK FK → shows.show_id	
seat_type_id	tinyint unsigned	NOT NULL	PK FK → seat_type.seat_type_id	
price	decimal(8,2)	NOT NULL		

Functional dependencies: (show_id, seat_type_id) → price

Candidate keys: {show_id, seat_type_id}

Example rows

show_id	seat_type_id	price
25	1	180.00
25	2	250.00
25	3	450.00
34	1	330.00
34	2	400.00
34	3	600.00

3.15 users 5 rows in sample DB

A customer account.

Attribute	Type	Null	Keys	Default / extra
user_id	bigint unsigned	NOT NULL	PK	auto_increment
full_name	varchar(120)	NOT NULL		
email	varchar(190)	NOT NULL	UQ	
phone	varchar(15)	NOT NULL	UQ	
created_at	datetime	NOT NULL		default CURRENT_TIMESTAMP

Functional dependencies: user_id → full_name, email, phone, created_at ; email → user_id ; phone → user_id

Candidate keys: {user_id}, {email}, {phone}

Example rows

user_id	full_name	email	phone
1	Ananya Rao	ananya.rao@example.com	9800000001
2	Rahul Verma	rahul.verma@example.com	9800000002
3	Priya Nair	priya.nair@example.com	9800000003
4	Karthik Reddy	karthik.reddy@example.com	9800000004

3.16 booking 3 rows in sample DB

A customer's booking. Starts as a timed hold (PENDING + hold_expires_at) and becomes CONFIRMED when payment is captured in time.

Attribute	Type	Null	Keys	Default / extra
booking_id	bigint unsigned	NOT NULL	PK UQ	auto_increment

Attribute	Type	Null	Keys	Default / extra
user_id	bigint unsigned	NOT NULL	FK → users.user_id UQ	
show_id	bigint unsigned	NOT NULL	FK → shows.show_id UQ	
status	enum('PENDING','CONFIRMED','EXPIRED','CANCELLED')	NOT NULL		default PENDING
idempotency_key	char(36)	NOT NULL	UQ	
hold_expires_at	datetime(3)	NOT NULL		
created_at	datetime(3)	NOT NULL		default CURRENT_TIMESTAMP(3)
confirmed_at	datetime(3)	NULL		

Functional dependencies: booking_id → user_id, show_id, status, idempotency_key, hold_expires_at, created_at, confirmed_at
; (user_id, idempotency_key) → booking_id

Candidate keys: {booking_id}, {user_id, idempotency_key}

Example rows

booking_id	user_id	show_id	status	idempotency_key	hold_expires_at	confirmed_at
1	1	25	CONFIRMED	6f1c1b4e-2a51-4a4e-9a0e-1f6d9a1b0001	2026-09-28 22:50:43.867000	2026-09-28 22:43:43.867000
2	2	25	PENDING	6f1c1b4e-2a51-4a4e-9a0e-1f6d9a1b0002	2026-09-28 23:50:43.867000	NULL
3	3	25	EXPIRED	6f1c1b4e-2a51-4a4e-9a0e-1f6d9a1b0003	2026-09-28 23:20:43.867000	NULL

3.17 booking_seat 6 rows in sample DB

Seats requested by a booking, with the price charged at booking time (historical snapshot).

Attribute	Type	Null	Keys	Default / extra
booking_id	bigint unsigned	NOT NULL	PK FK → booking.booking_id	
seat_id	bigint unsigned	NOT NULL	PK FK → seat.seat_id	
price	decimal(8,2)	NOT NULL		

Functional dependencies: (booking_id, seat_id) → price

Candidate keys: {booking_id, seat_id}

Example rows

booking_id	seat_id	price
1	75	450.00
1	76	450.00
2	45	250.00
2	46	250.00
2	47	250.00
3	1	180.00

3.18 show_seat 11760 rows in sample DB

The concurrency guard. Exactly one row per (show, seat). booking_id is the booking that currently owns the seat (NULL = free). The primary key makes a second owner impossible.

Attribute	Type	Null	Keys	Default / extra
show_id	bigint unsigned	NOT NULL	PK FK → booking.show_id FK → shows.show_id	
seat_id	bigint unsigned	NOT NULL	PK FK → seat.seat_id	
booking_id	bigint unsigned	NULL	FK → booking.booking_id	
version	int unsigned	NOT NULL		default 0

Functional dependencies: (show_id, seat_id) → booking_id, version

Candidate keys: {show_id, seat_id}

Example rows

show_id	seat_id	booking_id	version
25	1	NULL	0
25	45	2	1
25	74	NULL	0
25	75	1	1
25	76	1	1

3.19 payment 2 rows in sample DB

A payment attempt against a booking at a gateway.

Attribute	Type	Null	Keys	Default / extra
payment_id	bigint unsigned	NOT NULL	PK	auto_increment
booking_id	bigint unsigned	NOT NULL	FK → booking.booking_id	
gateway	varchar(20)	NOT NULL	UQ	
gateway_order_id	varchar(64)	NOT NULL	UQ	
gateway_payment_id	varchar(64)	NULL	UQ	
amount	decimal(10,2)	NOT NULL		
currency	char(3)	NOT NULL		default INR
status	enum('CREATED', 'CAPTURED', 'FAILED', 'REFUND_PENDING', 'REFUNDED')	NOT NULL		default CREATED
created_at	datetime(3)	NOT NULL		default CURRENT_TIMESTAMP(3)
updated_at	datetime(3)	NOT NULL		on update CURRENT_TIMESTAMP(3) · default CURRENT_TIMESTAMP(3)

Functional dependencies: payment_id → booking_id, gateway, gateway_order_id, gateway_payment_id, amount, currency, status, created_at, updated_at ; (gateway, gateway_order_id) → payment_id ; (gateway, gateway_payment_id) → payment_id

Candidate keys: {payment_id}, {gateway, gateway_order_id}, {gateway, gateway_payment_id}

Example rows

payment_id	booking_id	gateway	gateway_order_id	gateway_payment_id	amount	currency	status
1	1	RAZORPAY	order_Q1A2B3C4D5	pay_Q9Z8Y7X6W5	900.00	INR	CAPTURED
2	2	RAZORPAY	order_Q1A2B3C4D6	NULL	750.00	INR	CREATED

3.20 payment_webhook_event 1 rows in sample DB

Every webhook delivered by a payment gateway, stored once. The primary key on the gateway's event id is what makes webhook processing idempotent.

Attribute	Type	Null	Keys	Default / extra
gateway	varchar(20)	NOT NULL	PK	
event_id	varchar(64)	NOT NULL	PK	
event_type	varchar(40)	NOT NULL		
payment_id	bigint unsigned	NULL	FK → payment.payment_id	
payload	json	NOT NULL		
outcome	varchar(40)	NULL		
received_at	datetime(3)	NOT NULL		default CURRENT_TIMESTAMP(3)
processed_at	datetime(3)	NULL		

Functional dependencies: (gateway, event_id) → event_type, payment_id, payload, outcome, received_at, processed_at

Candidate keys: {gateway, event_id}

Example rows

gateway	event_id	event_type	payment_id	payload	outcome	processed_at
RAZORPAY	evt_Qa1b2c3d4e5	payment.captured	1	{"amount": 90000, "currency": "INR", &qu...	BOOKING_CONFIRMED	2026-09-28 22:43:43.880000

4. Normalisation: 1NF, 2NF, 3NF, BCNF

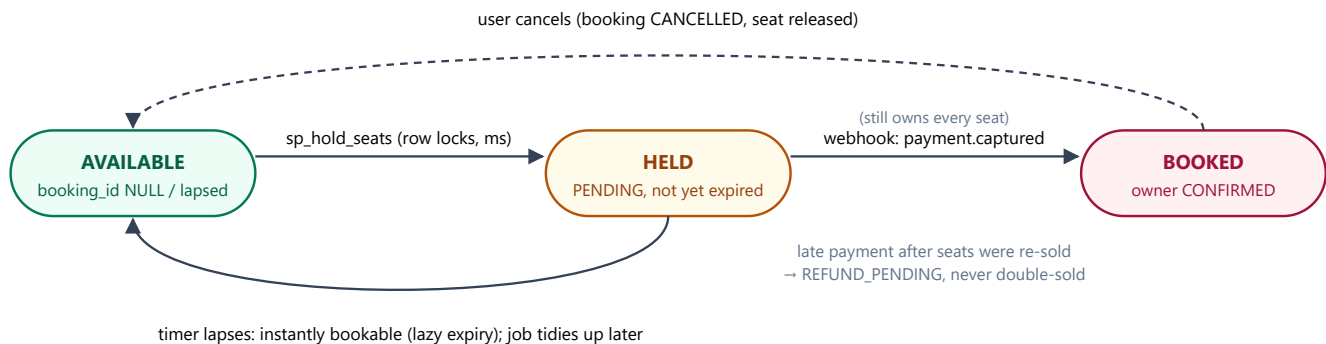
Form	Rule	How the schema satisfies it
1NF	Atomic values, no repeating groups, every table has a key.	Every table has a primary key. Multi-valued facts are separate rows, not lists: a movie's genres live in <code>movie_genre</code> , a theatre's amenities in <code>theatre_amenity</code> , a booking's seats in <code>booking_seat</code> . No comma-separated columns. (The webhook payload is stored as an opaque audit blob and never queried into, so it is a single value from the database's point of view.)
2NF	1NF, and no non-key attribute depends on only <i>part</i> of a composite key.	Composite-key tables: <code>show_price</code> (<code>show_id</code> , <code>seat_type_id</code>) → <code>price</code> , <code>booking_seat</code> (<code>booking_id</code> , <code>seat_id</code>) → <code>price</code> , <code>show_seat</code> (<code>show_id</code> , <code>seat_id</code>) → <code>booking_id</code> , <code>version</code> . Each non-key attribute needs the whole key. <code>show_id</code> was deliberately <i>removed</i> from <code>booking_seat</code> because <code>booking_id</code> → <code>show_id</code> alone (a partial dependency).
3NF	2NF, and no transitive dependency (non-key → non-key).	<code>theatre_id</code> is <i>not</i> stored on <code>shows</code> (<code>screen_id</code> → <code>theatre_id</code>); <code>price</code> is <i>not</i> stored on <code>show_seat</code> (<code>seat</code> → <code>seat_type</code> → <code>price</code>); a seat's AVAILABLE/HELD/BOOKED state is <i>not</i> stored (it is determined by the owner booking's status); derived values — booking total, screen capacity, show end time — are computed, not stored; a PIN code column was avoided (<code>PIN</code> → <code>city</code> would be a non-key determinant).
BCNF	For every non-trivial FD $X \rightarrow Y$, X is a superkey.	The FDs listed for each table in §3 show that every determinant is a candidate key (e.g. in <code>seat</code> both <code>seat_id</code> and (<code>screen_id</code> , <code>row_label</code> , <code>seat_number</code>) are keys; in <code>shows</code> , (<code>screen_id</code> , <code>start_time</code>) is a key because a screen starts one show at a time). No table has a determinant that is not a key, so all 20 tables are in BCNF.

Two design choices that look redundant but are not

- `booking_seat.price` is the price *agreed at booking time*. `show_price` may change later (dynamic pricing); the customer's receipt must not. It is a fact about the booking line, determined by (`booking_id`, `seat_id`) — not a copy of `show_price`.
- `show_seat.booking_id` vs `booking_seat` record different facts: `booking_seat` is the history of every seat any booking ever *requested* (including expired ones); `show_seat.booking_id` is the single *current* owner. Because (`show_id`, `seat_id`) is the primary key of `show_seat`, the database itself — not application code — enforces "one owner per seat". The composite foreign key (`booking_id`, `show_id`) → `booking`(`booking_id`, `show_id`) guarantees that owner belongs to the same show.

5. Concurrency and locking strategy

5.1 Seat lifecycle



A seat's state is **derived** from `show_seat.booking_id` and the owner booking's status / `hold_expires_at`. The 10-minute "hold" is **data**, not a database lock: row locks are held only for the few milliseconds a procedure runs, never while the customer is on the payment page.

5.2 Threats and the mechanism that stops each one

Threat	Mechanism
Two users grab the same seat at the same instant	<code>sp_hold_seats</code> takes <code>SELECT ... FOR UPDATE</code> on each <code>show_seat</code> row by primary key. The second transaction waits, then its locking read sees the committed owner and returns <code>SEATS_UNAVAILABLE</code> . Locking reads always read the latest committed version, so a stale snapshot can never make a taken seat look free.
A multi-seat request half-succeeds	All seats are locked and checked inside one transaction; any unavailable seat rolls back the whole request (all-or-nothing).
Deadlocks between concurrent transactions	(1) Every procedure acquires row locks in one global order — <code>show_seat</code> rows in ascending <code>seat_id</code> , then booking rows — so row locks cannot form a wait cycle. (2) Each procedure runs its transaction at <code>READ COMMITTED</code> , which switches off InnoDB gap locks; correctness relies only on primary-key row locks and unique keys, never on gap locks. The load test (§9) counts every deadlock: 0 across all scenarios.
Hold never released (lost inventory)	Lazy expiry : a <code>PENDING</code> booking whose <code>hold_expires_at</code> has passed counts as free the moment it lapses — no job needs to run. The <code>sp_release_expired_holds</code> job (MySQL EVENT every 30 s) only tidies up; correctness does not depend on it.
Hold stolen while the customer pays (lost hold)	A live hold (not expired) is never treated as free. The integrity check "live hold not owning all its seats" returns 0 after every scenario.
Payment arrives after the hold lapsed	The webhook locks the seats and confirms only if the booking <i>still owns every seat</i> (nobody took them — a good customer experience). Otherwise it releases what is left and marks the payment <code>REFUND_PENDING</code> . A taken-over hold is marked <code>EXPIRED</code> by the new buyer's transaction, so it can never be confirmed afterwards.
Duplicate / retried webhooks	<code>payment_webhook_event</code> has PRIMARY KEY (<code>gateway</code> , <code>event_id</code>). The first delivery inserts and processes; concurrent duplicates block on the key and then get <code>DUPLICATE_EVENT</code> . A different event for an already-captured payment is ignored by the payment state machine (only <code>CREATED/FAILED</code> → <code>CAPTURED</code>).
Double-click on "Pay" / client retry	<code>UNIQUE (user_id, idempotency_key)</code> on booking: the retry returns the original booking (<code>ALREADY_HELD</code>).
Client sends a fake amount	<code>sp_create_payment</code> computes the amount from <code>booking_seat</code> ; the webhook rejects a mismatched amount (<code>AMOUNT_MISMATCH</code> → refund).

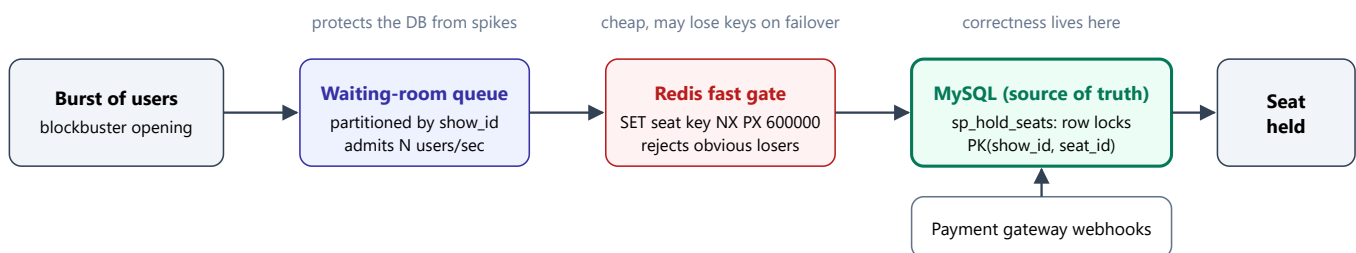
5.3 Hold transaction (core of sp_hold_seats)

```
START TRANSACTION;
INSERT INTO booking (user_id, show_id, status, idempotency_key, hold_expires_at)
VALUES (?, ?, 'PENDING', ?, NOW(3) + INTERVAL 600 SECOND);           -- new row: nobody waits on it

-- for each requested seat, in ascending seat_id order:
SELECT ss.booking_id, b.status, b.hold_expires_at
FROM show_seat ss LEFT JOIN booking b ON b.booking_id = ss.booking_id
WHERE ss.show_id = ? AND ss.seat_id = ?                               -- full PK: locks exactly one row
FOR UPDATE;
-- free = owner IS NULL OR owner EXPIRED/CANCELLED OR (PENDING AND hold_expires_at <= NOW(3))
-- not free -> ROLLBACK, return SEATS_UNAVAILABLE
UPDATE show_seat SET booking_id = @new_booking, version = version + 1
WHERE show_id = ? AND seat_id = ?;
INSERT INTO booking_seat ... ;                                         -- price snapshot
COMMIT;
```

Every statement addresses show_seat by its full primary key, so InnoDB locks only the requested rows. Users picking *different* seats of the same show proceed in parallel; only users colliding on the same seat serialise — and only for milliseconds.

5.4 At BookMyShow scale: Redis gate and a queue for bursts



- **Waiting-room queue** (Kafka / SQS, partitioned by show_id): on a blockbuster release the front door admits users at a rate the database can serve and shows everyone else their position, instead of letting a spike exhaust connections.
- **Redis fast gate**: SET hold:{show}:{seat} <booking> NX PX 600000 (a Lua script for all-or-nothing multi-seat holds) answers "already taken" in microseconds, so most losers never reach MySQL; the key's TTL gives the timed release for free.
- **MySQL remains the source of truth**. Redis can lose keys on failover, so a Redis "yes" is always confirmed by sp_hold_seats. If Redis is down the system is slower, never incorrect — which is exactly why the database layer in this submission is complete on its own.

6. Pessimistic vs optimistic locking

	Pessimistic (chosen for holds)	Optimistic (version compare-and-set)
Mechanism	SELECT ... FOR UPDATE, then write	Read version; UPDATE ... WHERE version = ?; 0 rows ⇒ lost, roll back
Under low contention	Small lock overhead	Cheapest — no lock held between read and write
Under high contention (hot seats)	Losers wait briefly, then get a clean “unavailable” before doing any work	Every loser does the full transaction, then throws it away (wasted work grows with contention)
Measured on the 500-user stampede (§9)	p50 30.3 ms , p95 46.4 ms · wasted transactions: 0	p50 110.2 ms , p95 124.2 ms · wasted transactions: 499
Correctness	1 winner, 0 double bookings	1 winner, 0 double bookings
Best used for	Seat holds on popular shows (hot rows)	Low-contention edits: show/price admin updates, profile edits

Both strategies are correct; the difference is cost under contention. Seat selection is the textbook hot-spot workload, so holds use pessimistic row locks — kept short (milliseconds) and ordered (deadlock-free). The version column on show_seat is still maintained so optimistic readers (e.g. a cached seat map) can detect change cheaply.

7. Indexing and query optimisation

Table	Index	Columns	Serves
screen	uq_screen_theatre_name	(theatre_id, name)	P2: theatre → its screens (covering)
shows	uq_show_screen_start	(screen_id, start_time)	P2: screen + day range scan; one show per screen per start time
shows	idx_show_movie_start	(movie_id, start_time)	“Where is this movie playing?” (movie page)
booking	uq_booking_idempotency	(user_id, idempotency_key)	Idempotent hold requests (retry returns same booking)
booking	idx_booking_status_expiry	(status, hold_expires_at)	Clean-up job finds expired PENDING holds
booking	uq_booking_id_show	(booking_id, show_id)	Target of show_seat composite FK (owner belongs to same show)
show_seat	PRIMARY	(show_id, seat_id)	One owner per seat per show; all seat locks are PK point locks
show_seat	idx_show_seat_booking	(booking_id, show_id)	Release / confirm all seats of a booking
payment	uq_payment_gateway_order	(gateway, gateway_order_id)	Webhook → payment lookup; one order id per gateway
payment	uq_payment_gateway_payment	(gateway, gateway_payment_id)	Gateway payment id can never be recorded twice
payment_webhook_event	PRIMARY	(gateway, event_id)	Duplicate webhook deliveries rejected (idempotency)

Table	Index	Columns	Serves
seat	uq_seat_position	(screen_id, row_label, seat_number)	No two seats with the same row/number in a screen
theatre	uq_theatre_city_name	(city_id, name)	Theatres of a city; no duplicate theatre names in a city

- **SARGable date filter.** P2 uses `start_time >= @d AND start_time < @d + INTERVAL 1 DAY` instead of `DATE(start_time) = @d`; wrapping the column in a function would stop MySQL from using `uq_show_screen_start`.
- **Normalised but cheap joins.** Omitting `theatre_id` from shows costs one join through screen, but a theatre has only a handful of screens, and that join is a covering index lookup.
- **Point locks only.** All locking statements use full primary keys; no range locks on the seat inventory.
- **Growth plan.** `show_seat` grows by (shows × seats) — the largest table. Rows for finished shows are archived; beyond a single node, shard by `show_id` (every hot transaction touches exactly one show).

8. P2 – list all shows on a given date at a given theatre

Inputs: @theatre_id and @show_date. Cancelled shows are excluded. Two shapes are provided: a flat list (one row per show) and the UI shape (one card per movie/language/format with its timings).

```
SET @theatre_id = 1;
SET @show_date = CURDATE() + INTERVAL 1 DAY;    -- e.g. DATE('2026-09-29')

SELECT t.name AS theatre,
       m.title AS movie,
       m.certificate,
       l.name AS language,
       f.name AS format,
       sc.name AS screen,
       s.show_id,
       TIME_FORMAT(s.start_time, '%h:%i %p') AS show_time,
       s.cancellation_allowed AS cancellable
FROM theatre t
      JOIN screen sc ON sc.theatre_id = t.theatre_id
      JOIN shows s ON s.screen_id = sc.screen_id
      JOIN movie m ON m.movie_id = s.movie_id
      JOIN language l ON l.language_id = s.language_id
      JOIN show_format f ON f.format_id = s.format_id
WHERE t.theatre_id = @theatre_id
      AND s.start_time >= @show_date
      AND s.start_time < @show_date + INTERVAL 1 DAY
      AND s.status = 'SCHEDULED'
ORDER BY m.title, l.name, f.name, s.start_time;
```

Output (theatre 1 = Aurora Cinemas: Kukatpally, tomorrow)

theatre	movie	certificate	language	format	screen	show_id	show_time	cancellable
Aurora Cinemas: Kukatpally	Chasing Tides	U	Telugu	2D	Audi 2	27	12:00 PM	1
Aurora Cinemas: Kukatpally	Iron Valley	UA	Hindi	IMAX 2D	IMAX	32	02:30 PM	1
Aurora Cinemas: Kukatpally	Iron Valley	UA	Telugu	2D	Audi 1	22	10:00 AM	1
Aurora Cinemas: Kukatpally	Iron Valley	UA	Telugu	2D	Audi 1	23	01:45 PM	1
Aurora Cinemas: Kukatpally	Iron Valley	UA	Telugu	2D	Audi 1	25	09:15 PM	1
Aurora Cinemas: Kukatpally	Iron Valley	UA	Telugu	IMAX 2D	IMAX	31	11:00 AM	1
Aurora Cinemas: Kukatpally	Iron Valley	UA	Telugu	IMAX 2D	IMAX	33	06:15 PM	1
Aurora Cinemas: Kukatpally	Little Astronaut	U	English	2D	Audi 2	26	09:30 AM	1
Aurora Cinemas: Kukatpally	Monsoon Heist	UA	Hindi	2D	Audi 1	24	05:30 PM	1
Aurora Cinemas: Kukatpally	Monsoon Heist	UA	Telugu	2D	Audi 2	28	03:00 PM	1
Aurora Cinemas: Kukatpally	The Midnight Ledger	A	English	2D	Audi 2	29	06:30 PM	1
Aurora Cinemas: Kukatpally	The Midnight Ledger	A	English	IMAX 2D	IMAX	34	10:00 PM	1

UI shape – one card per movie / language / format

```
SELECT m.title AS movie, m.certificate, l.name AS language, f.name AS format,
       COUNT(*) AS shows,
       GROUP_CONCAT(TIME_FORMAT(s.start_time, '%h:%i %p')
                    ORDER BY s.start_time SEPARATOR ' | ') AS show_timings
FROM screen sc
      JOIN shows s ON s.screen_id = sc.screen_id
      JOIN movie m ON m.movie_id = s.movie_id
      JOIN language l ON l.language_id = s.language_id
      JOIN show_format f ON f.format_id = s.format_id
WHERE sc.theatre_id = @theatre_id
      AND s.start_time >= @show_date
      AND s.start_time < @show_date + INTERVAL 1 DAY
```

```

AND s.status = 'SCHEDULED'
GROUP BY m.movie_id, m.title, m.certificate, l.name, f.name
ORDER BY m.title, l.name, f.name;

```

movie	certificate	language	format	shows	show_timings
Chasing Tides	U	Telugu	2D	1	12:00 PM
Iron Valley	UA	Hindi	IMAX 2D	1	02:30 PM
Iron Valley	UA	Telugu	2D	3	10:00 AM 01:45 PM 09:15 PM
Iron Valley	UA	Telugu	IMAX 2D	2	11:00 AM 06:15 PM
Little Astronaut	U	English	2D	1	09:30 AM
Monsoon Heist	UA	Hindi	2D	1	05:30 PM
Monsoon Heist	UA	Telugu	2D	1	03:00 PM
The Midnight Ledger	A	English	2D	1	06:30 PM
The Midnight Ledger	A	English	IMAX 2D	1	10:00 PM

The sample data contains a cancelled 10:00 PM show in Audi 2 on this date; it is correctly absent.

Supporting query – the 7-day date strip

show_date	label	shows
2026-09-28	Mon 28 Sep	13
2026-09-29	Tue 29 Sep	12
2026-09-30	Wed 30 Sep	13
2026-10-01	Thu 01 Oct	13
2026-10-02	Fri 02 Oct	13
2026-10-03	Sat 03 Oct	13
2026-10-04	Sun 04 Oct	13

Execution plan (EXPLAIN)

i d	select _type	tab le	partit ions	typ e	possible_keys	key	key_ len	ref	ro ws	filte red	Extra
1	SIMPLE	sc	NULL	ref	PRIMARY,uq_screen_theatr e_name	uq_screen_the atre_name	4	const	3	100.0 0	Using index
1	SIMPLE	s	NULL	ref	uq_show_screen_start,idx _show_movie_start	uq_show_scre en_start	4	bookmyshow.sc .screen_id	29	5.55	Using index condition; Using where
1	SIMPLE	m	NULL	eq_ ref	PRIMARY	PRIMARY	4	bookmyshow.s. movie_id	1	100.0 0	NULL

The plan reads the theatre's screens from the covering index `uq_screen_theatre_name`, then, per screen, uses `uq_show_screen_start` with index condition push-down on `start_time`, and finally fetches each movie by primary key (`eq_ref`). No table is scanned.

9. Load and concurrency test

load_test/load_test.py drives the stored procedures with many truly simultaneous clients (one MySQL connection per thread, all released together behind a barrier) and then checks the invariants **by querying the database**, not by trusting return values.

500

simulated users per scenario

100

concurrent DB connections

0

double-booked seats, lost holds, or deadlocks

5/5

scenarios passed

Scenario	Outcomes	Invariant checks (from DB)	p50 / p95 / p99 ms	Result
S1 Hot-seat stampede (pessimistic) 500 users, same 2 seats (C5,C6), released together	SEATS_UNAVAILABLE: 499 HELD: 1	deadlocks or lock timeouts: 0 double owned seats: 0 lost holds: 0	30.3 / 46.4 / 57.6	PASS
S2 Sold-out rush 500 users, random 1-4 seat blocks, 80-seat show	SEATS_UNAVAILABLE: 452 HELD: 48	deadlocks or lock timeouts: 0 double owned seats: 0 lost holds: 0 seats sold: 76 capacity: 80	36.9 / 68.4 / 79.2	PASS
S3 Webhook storm (idempotency) 50 paid bookings, each webhook delivered 5x concurrently (250 calls)	DUPLICATE_EVENT: 200 BOOKING_CONFIRMED: 50	deadlocks or lock timeouts: 0 bookings confirmed: 50 payments captured: 50 webhook rows stored: 50	31.7 / 65.0 / 74.2	PASS
S4 Expiry race (lapsed hold vs new buyer vs late payment) 30 seats held for 2 s, then new buyers + late payments fired together	pay:LATE_PAYMENT_REFUND: 13 pay:BOOKING_CONFIRMED: 17 buy:SEATS_UNAVAILABLE: 17 buy:HELD: 13	deadlocks or lock timeouts: 0 double owned seats: 0 lost holds: 0 seats without exactly one owner: 0	—	PASS
S5 Hot-seat stampede (optimistic CAS, for comparison) 500 users, same 2 seats, version compare-and-set	CONFLICT_ROLLED_BACK: 499 HELD: 1	deadlocks or lock timeouts: 0 double owned seats: 0 lost holds: 0 wasted transactions: 499	110.2 / 124.2 / 144.0	PASS

Environment: MySQL 8.0.46 (server default isolation REPEATABLE-READ; the procedures run their own transactions at READ-COMMITTED), single developer laptop (Windows 11), client and server on the same machine; run at 2026-09-28 23:40:45. Latencies are per call, including lock waits. Raw results: load_test/results.json and results.txt.

Findings fixed because of the load test. The first runs passed every invariant but logged deadlocks (MySQL error 1213) in S4. InnoDB rolled one transaction back each time, which is safe (the webhook's event row rolled back with it, so a gateway retry succeeds). A deadlock is still a failed request, though, so both causes were traced with `SHOW ENGINE INNODB STATUS` and fixed:

1. **Gap locks.** Under the default `REPEATABLE READ`, releasing "all seats of booking X" range-scans the secondary index `idx_show_seat_booking` and also locks the *gap* to the next entry. That blocks an unrelated buyer from writing a new owner. Fix: each procedure's transaction runs at `READ COMMITTED`, which has no gap locks. Correctness never depended on gap locks.
2. **A hidden locking read.** In the webhook, `SET v_show_id = (SELECT show_id FROM booking ...)` is a *locking* read inside a stored procedure. It took a shared lock on the booking row *before* the seat locks, which reverses the global lock order. Fix: a plain `SELECT ... INTO`, which does not lock.

After both fixes, **six consecutive full runs recorded 0 deadlocks**. The numbers below are from the final run.

What each scenario proves

- **S1 Hot-seat stampede** — the worst case: everyone wants the same seats at the same instant. Exactly one hold; everyone else is rejected cleanly (no errors, no timeouts).
- **S2 Sold-out rush** — overlapping random multi-seat requests on one show. Seats sold equals owned inventory rows, never above capacity, no seat has two owners, and 0 deadlocks despite overlapping seat sets.
- **S3 Webhook storm** — each of 50 payment webhooks delivered 5 times concurrently: 50 confirmations, 200 duplicates ignored, exactly one stored event per payment.
- **S4 Expiry race** — holds lapse; new buyers and the old buyers' late payments hit the same seats at the same moment. Whichever arrives first wins; every seat ends with exactly one owner; the loser of each race is refunded.
- **S5 Optimistic comparison** — same stampede with version compare-and-set: also correct, but every loser does and discards a full transaction (499 wasted), and median latency rises from 30.3 ms to 110.2 ms — the data behind choosing pessimistic locking for holds (§6).

10. How to run

```
-- MySQL 8.0.4+ ; run from the submission folder
mysql -u root -p < sql/01_schema.sql                -- P1: tables
mysql -u root -p bookmyshow < sql/02_procedures.sql  -- P1: Locking procedures
mysql -u root -p bookmyshow < sql/03_sample_data.sql -- P1: sample rows
mysql -u root -p bookmyshow < sql/04_p2_shows_by_theatre_and_date.sql -- P2
mysql -u root -p bookmyshow < sql/05_demo_and_checks.sql -- walk-through + integrity checks

-- or in MySQL Workbench: open sql/bookmyshow_complete.sql (01-04 combined) and execute

-- Load test (test database only)
pip install -r load_test/requirements.txt
python load_test/load_test.py --user root --password <pwd> --users 500 --workers 100
```

Appendix A – P1 SQL: schema (sql/01_schema.sql)

```
-- =====
-- BookMyShow-style Ticketing Backend | P1 - Schema (DDL)
-- Target : MySQL 8.0+ (InnoDB, utf8mb4)
-- Run    : mysql -u root -p < sql/01_schema.sql
--
-- Design goals
-- * 1NF / 2NF / 3NF / BCNF: every non-key attribute depends on the key, the
--   whole key and nothing but the key; every determinant is a candidate key.
-- * No double booking: exactly ONE row per (show, seat) in show_seat. The
--   PRIMARY KEY makes a second owner physically impossible, and every write
--   to that row happens under an InnoDB row lock (see 02_procedures.sql).
-- * No lost holds: a hold's expiry lives in the booking row, so it can never
--   be forgotten; expired holds are free for others instantly (lazy expiry)
--   even if the clean-up job is down.
-- ALL DATETIME values are stored in IST (Asia/Kolkata), the theatre's local time.
-- =====

DROP DATABASE IF EXISTS bookmyshow;
CREATE DATABASE bookmyshow CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE bookmyshow;

-- -----
-- 1. Location & venue
-- -----

CREATE TABLE city (
    city_id    SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name       VARCHAR(80)        NOT NULL,
    state      VARCHAR(80)        NOT NULL,
    PRIMARY KEY (city_id),
    UNIQUE KEY uq_city_name_state (name, state)
) ENGINE = InnoDB;

CREATE TABLE theatre (
    theatre_id INT UNSIGNED      NOT NULL AUTO_INCREMENT,
    city_id    SMALLINT UNSIGNED NOT NULL,
    name       VARCHAR(120)      NOT NULL,
    address_line VARCHAR(255)    NOT NULL,
    created_at DATETIME          NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (theatre_id),
    UNIQUE KEY uq_theatre_city_name (city_id, name),
    CONSTRAINT fk_theatre_city FOREIGN KEY (city_id) REFERENCES city (city_id)
) ENGINE = InnoDB;

-- Theatre facilities shown on the Listing page (M-Ticket, Food & Beverage ...).
-- A separate table instead of a comma-separated column keeps 1NF.
CREATE TABLE amenity (
    amenity_id SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name       VARCHAR(60)        NOT NULL,
    PRIMARY KEY (amenity_id),
    UNIQUE KEY uq_amenity_name (name)
) ENGINE = InnoDB;

CREATE TABLE theatre_amenity (
    theatre_id INT UNSIGNED      NOT NULL,
    amenity_id SMALLINT UNSIGNED NOT NULL,
    PRIMARY KEY (theatre_id, amenity_id),
    KEY idx_theatre_amenity_amenity (amenity_id),
    CONSTRAINT fk_ta_theatre FOREIGN KEY (theatre_id) REFERENCES theatre (theatre_id),
    CONSTRAINT fk_ta_amenity FOREIGN KEY (amenity_id) REFERENCES amenity (amenity_id)
) ENGINE = InnoDB;

CREATE TABLE screen (
    screen_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
    theatre_id INT UNSIGNED NOT NULL,
```

```

    name          VARCHAR(40) NOT NULL,                -- 'Audi 1', 'IMAX'
    PRIMARY KEY (screen_id),
    UNIQUE KEY uq_screen_theatre_name (theatre_id, name), -- also serves "screens of a theatre"
    CONSTRAINT fk_screen_theatre FOREIGN KEY (theatre_id) REFERENCES theatre (theatre_id)
) ENGINE = InnoDB;

CREATE TABLE seat_type (
    seat_type_id TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name          VARCHAR(30) NOT NULL,                -- CLASSIC, PRIME, RECLINER
    PRIMARY KEY (seat_type_id),
    UNIQUE KEY uq_seat_type_name (name)
) ENGINE = InnoDB;

-- The physical seat. Capacity of a screen is COUNT(seat); it is not stored
-- (a stored copy would be a derived, update-anomaly-prone value).
CREATE TABLE seat (
    seat_id       BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    screen_id     INT UNSIGNED NOT NULL,
    row_label     VARCHAR(2) NOT NULL,                -- 'A' .. 'H'
    seat_number   TINYINT UNSIGNED NOT NULL,          -- 1 .. n
    seat_type_id  TINYINT UNSIGNED NOT NULL,
    PRIMARY KEY (seat_id),
    UNIQUE KEY uq_seat_position (screen_id, row_label, seat_number),
    KEY idx_seat_type (seat_type_id),
    CONSTRAINT fk_seat_screen FOREIGN KEY (screen_id) REFERENCES screen (screen_id),
    CONSTRAINT fk_seat_type FOREIGN KEY (seat_type_id) REFERENCES seat_type (seat_type_id)
) ENGINE = InnoDB;

-----
-- 2. Catalogue
-----

CREATE TABLE movie (
    movie_id      INT UNSIGNED NOT NULL AUTO_INCREMENT,
    title         VARCHAR(150) NOT NULL,
    duration_min  SMALLINT UNSIGNED NOT NULL,
    certificate   ENUM ('U', 'UA', 'A', 'S') NOT NULL,
    release_date  DATE NOT NULL,
    PRIMARY KEY (movie_id),
    UNIQUE KEY uq_movie_title_release (title, release_date),
    CONSTRAINT chk_movie_duration CHECK (duration_min BETWEEN 1 AND 600)
) ENGINE = InnoDB;

CREATE TABLE genre (
    genre_id      TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name          VARCHAR(40) NOT NULL,
    PRIMARY KEY (genre_id),
    UNIQUE KEY uq_genre_name (name)
) ENGINE = InnoDB;

CREATE TABLE movie_genre (
    -- many-to-many; keeps genres atomic (1NF)
    movie_id      INT UNSIGNED NOT NULL,
    genre_id      TINYINT UNSIGNED NOT NULL,
    PRIMARY KEY (movie_id, genre_id),
    KEY idx_movie_genre_genre (genre_id),
    CONSTRAINT fk_mg_movie FOREIGN KEY (movie_id) REFERENCES movie (movie_id),
    CONSTRAINT fk_mg_genre FOREIGN KEY (genre_id) REFERENCES genre (genre_id)
) ENGINE = InnoDB;

CREATE TABLE language (
    language_id   TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name          VARCHAR(40) NOT NULL,
    PRIMARY KEY (language_id),
    UNIQUE KEY uq_language_name (name)
) ENGINE = InnoDB;

CREATE TABLE show_format (
    format_id     TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name          VARCHAR(30) NOT NULL,                -- 2D, 3D, IMAX 2D, 4DX

```

```

    PRIMARY KEY (format_id),
    UNIQUE KEY uq_show_format_name (name)
) ENGINE = InnoDB;

-----

-- 3. Shows (one screening of a movie on a screen at a time)
-- theatre_id is NOT stored here: it is determined by screen_id
-- (screen_id -> theatre_id), so storing it would be a transitive
-- dependency (3NF violation). P2 reaches the theatre through screen.
-- End time is derived (start_time + movie.duration_min), so not stored.
-----

CREATE TABLE shows (
  show_id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  movie_id         INT UNSIGNED NOT NULL,
  screen_id        INT UNSIGNED NOT NULL,
  language_id      TINYINT UNSIGNED NOT NULL,
  format_id        TINYINT UNSIGNED NOT NULL,
  start_time       DATETIME NOT NULL,
  status           ENUM ('SCHEDULED', 'CANCELLED') NOT NULL DEFAULT 'SCHEDULED',
  cancellation_allowed BOOLEAN NOT NULL DEFAULT TRUE,
  PRIMARY KEY (show_id),
  -- a screen cannot start two shows at the same moment; this index is also
  -- the access path for P2: (screen_id = ? AND start_time in [day, day+1))
  UNIQUE KEY uq_show_screen_start (screen_id, start_time),
  KEY idx_show_movie_start (movie_id, start_time), -- "where is movie X playing"
  KEY idx_show_language (language_id),
  KEY idx_show_format (format_id),
  CONSTRAINT fk_show_movie FOREIGN KEY (movie_id) REFERENCES movie (movie_id),
  CONSTRAINT fk_show_screen FOREIGN KEY (screen_id) REFERENCES screen (screen_id),
  CONSTRAINT fk_show_language FOREIGN KEY (language_id) REFERENCES language (language_id),
  CONSTRAINT fk_show_format FOREIGN KEY (format_id) REFERENCES show_format (format_id)
) ENGINE = InnoDB;

-- Price depends on (show, seat type) - not on the individual seat - so it gets
-- its own table (putting it on show_seat would be a partial dependency).
CREATE TABLE show_price (
  show_id          BIGINT UNSIGNED NOT NULL,
  seat_type_id     TINYINT UNSIGNED NOT NULL,
  price            DECIMAL(8, 2) NOT NULL,
  PRIMARY KEY (show_id, seat_type_id),
  KEY idx_show_price_type (seat_type_id),
  CONSTRAINT fk_sp_show FOREIGN KEY (show_id) REFERENCES shows (show_id),
  CONSTRAINT fk_sp_type FOREIGN KEY (seat_type_id) REFERENCES seat_type (seat_type_id),
  CONSTRAINT chk_show_price CHECK (price > 0)
) ENGINE = InnoDB;

-----

-- 4. Customers, bookings and the seat inventory
-----

CREATE TABLE users (
  user_id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  full_name        VARCHAR(120) NOT NULL,
  email            VARCHAR(190) NOT NULL,
  phone            VARCHAR(15) NOT NULL,
  created_at       DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (user_id),
  UNIQUE KEY uq_users_email (email),
  UNIQUE KEY uq_users_phone (phone)
) ENGINE = InnoDB;

-- A booking starts life as a timed HOLD (status PENDING + hold_expires_at) and
-- becomes CONFIRMED when the payment webhook arrives in time.
-- idempotency_key : generated by the client once per "Proceed" click, so a
-- double-click / network retry can never create two holds.
-- The total amount is NOT stored: it is SUM(booking_seat.price) (derived).
CREATE TABLE booking (
  booking_id       BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id          BIGINT UNSIGNED NOT NULL,

```

```

show_id          BIGINT UNSIGNED NOT NULL,
status           ENUM ('PENDING', 'CONFIRMED', 'EXPIRED', 'CANCELLED') NOT NULL DEFAULT 'PENDING',
idempotency_key  CHAR(36)          NOT NULL,
hold_expires_at  DATETIME(3)       NOT NULL,
created_at       DATETIME(3)       NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
confirmed_at     DATETIME(3)       NULL,
PRIMARY KEY (booking_id),
UNIQUE KEY uq_booking_idempotency (user_id, idempotency_key),
UNIQUE KEY uq_booking_id_show (booking_id, show_id),      -- target of show_seat's composite FK
KEY idx_booking_show (show_id),
KEY idx_booking_status_expiry (status, hold_expires_at), -- clean-up job
CONSTRAINT fk_booking_user FOREIGN KEY (user_id) REFERENCES users (user_id),
CONSTRAINT fk_booking_show FOREIGN KEY (show_id) REFERENCES shows (show_id)
) ENGINE = InnoDB;

-- Which seats a booking asked for, with the price charged at that moment
-- (a historical snapshot - later price changes must not alter old bookings).
-- show_id is deliberately absent: booking_id -> show_id already, so repeating it
-- here would be a partial dependency on part of the key (2NF violation).
CREATE TABLE booking_seat (
  booking_id  BIGINT UNSIGNED NOT NULL,
  seat_id     BIGINT UNSIGNED NOT NULL,
  price       DECIMAL(8, 2)    NOT NULL,
  PRIMARY KEY (booking_id, seat_id),
  KEY idx_booking_seat_seat (seat_id),
  CONSTRAINT fk_bs_booking FOREIGN KEY (booking_id) REFERENCES booking (booking_id),
  CONSTRAINT fk_bs_seat    FOREIGN KEY (seat_id)    REFERENCES seat (seat_id)
) ENGINE = InnoDB;

-- THE CONCURRENCY GUARD. One row per (show, seat), created when the show is
-- scheduled. booking_id = the booking that currently owns the seat (NULL = never
-- taken / released). Because (show_id, seat_id) is the PRIMARY KEY, a seat can
-- have at most one owner at any instant - double booking is structurally
-- impossible. The seat's state is derived, never stored twice:
--   AVAILABLE : booking_id IS NULL, or owner is EXPIRED/CANCELLED,
--               or owner is PENDING with hold_expires_at <= NOW()
--   HELD      : owner is PENDING and hold_expires_at > NOW()
--   BOOKED    : owner is CONFIRMED
-- The composite FK (booking_id, show_id) guarantees the owner booking belongs to
-- the SAME show. version supports optimistic (compare-and-set) updates.
CREATE TABLE show_seat (
  show_id     BIGINT UNSIGNED NOT NULL,
  seat_id     BIGINT UNSIGNED NOT NULL,
  booking_id  BIGINT UNSIGNED NULL,
  version     INT UNSIGNED     NOT NULL DEFAULT 0,
  PRIMARY KEY (show_id, seat_id),
  KEY idx_show_seat_booking (booking_id, show_id),
  KEY idx_show_seat_seat (seat_id),
  CONSTRAINT fk_ss_show    FOREIGN KEY (show_id) REFERENCES shows (show_id),
  CONSTRAINT fk_ss_seat    FOREIGN KEY (seat_id) REFERENCES seat (seat_id),
  CONSTRAINT fk_ss_booking FOREIGN KEY (booking_id, show_id) REFERENCES booking (booking_id, show_id)
) ENGINE = InnoDB;

-- -----
-- 5. Payments and idempotent webhooks
-- -----

CREATE TABLE payment (
  payment_id  BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  booking_id  BIGINT UNSIGNED NOT NULL,
  gateway     VARCHAR(20)      NOT NULL,      -- RAZORPAY, PAYTM ...
  gateway_order_id VARCHAR(64) NOT NULL,      -- created before redirecting the user
  gateway_payment_id VARCHAR(64) NULL,        -- known once the gateway reports back
  amount      DECIMAL(10, 2)   NOT NULL,
  currency    CHAR(3)          NOT NULL DEFAULT 'INR',
  status      ENUM ('CREATED', 'CAPTURED', 'FAILED', 'REFUND_PENDING', 'REFUNDED') NOT NULL DEFAULT
'CREATED',
  created_at  DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at  DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),

```

```

PRIMARY KEY (payment_id),
UNIQUE KEY uq_payment_gateway_order (gateway, gateway_order_id),
UNIQUE KEY uq_payment_gateway_payment (gateway, gateway_payment_id),
KEY idx_payment_booking (booking_id),
CONSTRAINT fk_payment_booking FOREIGN KEY (booking_id) REFERENCES booking (booking_id),
CONSTRAINT chk_payment_amount CHECK (amount > 0)
) ENGINE = InnoDB;

-- Every webhook delivery is recorded once. Gateways deliver "at least once"
-- (retries, duplicates, out-of-order); the PRIMARY KEY on the gateway's own
-- event id turns a duplicate delivery into a no-op (idempotency).
-- payload is kept verbatim as an audit record; it is never queried into.
CREATE TABLE payment_webhook_event (
  gateway      VARCHAR(20)    NOT NULL,
  event_id     VARCHAR(64)    NOT NULL,
  event_type   VARCHAR(40)    NOT NULL,          -- payment.captured, payment.failed
  payment_id   BIGINT UNSIGNED NULL,             -- resolved while processing
  payload      JSON           NOT NULL,
  outcome      VARCHAR(40)    NULL,              -- BOOKING_CONFIRMED, LATE_PAYMENT_REFUND ...
  received_at  DATETIME(3)    NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  processed_at DATETIME(3)    NULL,
  PRIMARY KEY (gateway, event_id),
  KEY idx_webhook_payment (payment_id),
  CONSTRAINT fk_webhook_payment FOREIGN KEY (payment_id) REFERENCES payment (payment_id)
) ENGINE = InnoDB;

```

Appendix B – P1 SQL: locking procedures (sql/02_procedures.sql)

```
-- =====
-- BookMyShow-style Ticketing Backend | P1 - Locking strategy (stored procedures)
-- Target : MySQL 8.0.4+ (uses JSON_TABLE). Run AFTER 01_schema.sql:
--         mysql -u root -p bookmyshow < sql/02_procedures.sql
--
-- Strategy: PESSIMISTIC row locks (SELECT ... FOR UPDATE) on show_seat, the
-- one-row-per-(show, seat) table, inside short transactions.
--
-- Deadlock prevention
-- 1. Every transaction acquires locks in the same order:
--     show_seat rows (ascending seat_id) -> booking row(s)
--     sp_hold_seats, sp_process_payment_webhook and sp_release_expired_holds
--     all follow it, so row locks can never wait on each other in a cycle.
-- 2. Each procedure runs its transaction at READ COMMITTED, which disables
--     InnoDB gap locks. Under REPEATABLE READ, a range update on a secondary
--     index (e.g. "release every seat of booking X") also locks the gap to the
--     next index entry and can block an unrelated insert -> deadlock.
-- 3. Reads that must NOT lock use plain SELECT ... INTO. Inside a procedure,
--     SET v = (SELECT ...) is a LOCKING read and silently breaks rule 1.
-- Rules 2 and 3 were both found by the load test (error 1213 in the expiry
-- race, diagnosed with SHOW ENGINE INNODB STATUS) and fixed.
--
-- Why pessimistic here (and not optimistic)? A popular show's front rows are
-- a HOT SPOT: hundreds of users click the same seats in the same second. With
-- optimistic version checks all but one of them do the work, fail at commit
-- and retry - wasted round-trips that grow with contention. A row lock makes
-- losers wait a few milliseconds and then get a clean "unavailable" answer.
-- Locks are held only for the few ms the procedure runs (never while the user
-- is paying) - the long 10-minute "hold" is DATA (hold_expires_at), not a lock.
-- An optimistic variant using show_seat.version is shown in 05_demo_and_checks.sql.
-- =====

USE bookmyshow;

DROP PROCEDURE IF EXISTS sp_hold_seats;
DROP PROCEDURE IF EXISTS sp_create_payment;
DROP PROCEDURE IF EXISTS sp_process_payment_webhook;
DROP PROCEDURE IF EXISTS sp_release_expired_holds;

DELIMITER $$

-- -----
-- sp_hold_seats: atomically hold 1..10 seats of a show for one user.
-- p_seat_ids      JSON array of seat_id, e.g. '[101,102]'
-- p_idempotency_key client-generated UUID; retrying with the same key returns
--                  the same booking instead of creating a second hold
-- p_hold_seconds  hold duration (600 = 10 minutes in production)
-- o_status        HELD | ALREADY_HELD | SEATS_UNAVAILABLE | INVALID_SEATS |
--                  SHOW_NOT_BOOKABLE | BAD_REQUEST
-- -----

CREATE PROCEDURE sp_hold_seats(
  IN p_user_id      BIGINT UNSIGNED,
  IN p_show_id      BIGINT UNSIGNED,
  IN p_seat_ids     JSON,
  IN p_idempotency_key CHAR(36),
  IN p_hold_seconds INT,
  OUT o_booking_id  BIGINT UNSIGNED,
  OUT o_status      VARCHAR(30))
proc:
BEGIN
  DECLARE v_seat_id  BIGINT UNSIGNED;
```

```

DECLARE v_found          TINYINT;
DECLARE v_owner          BIGINT UNSIGNED;
DECLARE v_owner_status   VARCHAR(20);
DECLARE v_owner_exp      DATETIME(3);
DECLARE v_requested      INT;
DECLARE v_done           TINYINT DEFAULT 0;

-- seats in ascending order => global lock order => no deadlocks between holds
DECLARE seat_cursor CURSOR FOR
    SELECT DISTINCT j.seat_id
    FROM JSON_TABLE(p_seat_ids, '$[*]' COLUMNS (seat_id BIGINT UNSIGNED PATH '$')) AS j
    ORDER BY j.seat_id;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

-- the same idempotency key raced in by a parallel retry: return that booking
DECLARE EXIT HANDLER FOR 1062
BEGIN
    ROLLBACK;
    SET o_booking_id = (SELECT booking_id FROM booking
                       WHERE user_id = p_user_id AND idempotency_key = p_idempotency_key);
    SET o_status = 'ALREADY_HELD';
END;
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    ROLLBACK;
    RESIGNAL;          -- e.g. lock wait timeout: caller may retry
END;

SET o_booking_id = NULL;

-- 1) Idempotent replay (no transaction needed, the key is unique)
SET o_booking_id = (SELECT booking_id FROM booking
                   WHERE user_id = p_user_id AND idempotency_key = p_idempotency_key);
IF o_booking_id IS NOT NULL THEN
    SET o_status = 'ALREADY_HELD';
    LEAVE proc;
END IF;

-- 2) Request validation
SET v_requested = (SELECT COUNT(DISTINCT j.seat_id)
                  FROM JSON_TABLE(p_seat_ids, '$[*]' COLUMNS (seat_id BIGINT UNSIGNED PATH '$')) AS j);
IF v_requested = 0 OR v_requested > 10 OR p_hold_seconds <= 0 THEN
    SET o_status = 'BAD_REQUEST';
    LEAVE proc;
END IF;

IF NOT EXISTS (SELECT 1 FROM shows
              WHERE show_id = p_show_id AND status = 'SCHEDULED' AND start_time > NOW()) THEN
    SET o_status = 'SHOW_NOT_BOOKABLE';
    LEAVE proc;
END IF;

-- READ COMMITTED: correctness comes from primary-key row locks and unique
-- keys, not from gap locks. RC turns gap locking off, which removes the
-- insert-vs-range-scan deadlocks that REPEATABLE READ produces under load.
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;

-- 3) Create the hold row first (it is new, so nobody else can be waiting on it).
--    A parallel retry with the same idempotency key fails here with 1062.
INSERT INTO booking (user_id, show_id, status, idempotency_key, hold_expires_at)
VALUES (p_user_id, p_show_id, 'PENDING', p_idempotency_key, NOW(3) + INTERVAL p_hold_seconds SECOND);
SET o_booking_id = LAST_INSERT_ID();

-- 4) Lock every requested seat (and its current owner booking) in seat order.
--    A locking read always sees the latest COMMITTED data, so two users can
--    never both see the same seat as free. Every statement below touches the
--    seat by its full primary key, so only the requested rows are locked -

```

```

-- other users keep booking other seats of the same show in parallel.
OPEN seat_cursor;
lock_loop:
LOOP
    FETCH seat_cursor INTO v_seat_id;
    IF v_done = 1 THEN
        LEAVE lock_loop;
    END IF;

    SET v_found = 0;
    SELECT 1, ss.booking_id, b.status, b.hold_expires_at
    INTO v_found, v_owner, v_owner_status, v_owner_exp
    FROM show_seat ss
        LEFT JOIN booking b ON b.booking_id = ss.booking_id
    WHERE ss.show_id = p_show_id
        AND ss.seat_id = v_seat_id
    FOR UPDATE;

    IF v_found = 0 THEN
        -- seat is not part of this show
        CLOSE seat_cursor;
        ROLLBACK;
        SET o_booking_id = NULL;
        SET o_status = 'INVALID_SEATS';
        LEAVE proc;
    END IF;

    IF NOT (v_owner IS NULL
        OR v_owner_status IN ('EXPIRED', 'CANCELLED')
        OR (v_owner_status = 'PENDING' AND v_owner_exp <= NOW(3))) THEN
        CLOSE seat_cursor;
        ROLLBACK;
        -- undoes the hold row, releases all locks
        SET o_booking_id = NULL;
        SET o_status = 'SEATS_UNAVAILABLE';
        LEAVE proc;
    END IF;

    -- a lapsed hold we are taking over is marked EXPIRED, so a late payment
    -- for it can never confirm (the webhook refunds it instead)
    IF v_owner_status = 'PENDING' THEN
        UPDATE booking SET status = 'EXPIRED'
        WHERE booking_id = v_owner AND status = 'PENDING';
    END IF;

    -- price snapshot for this seat (seat type -> show price)
    INSERT INTO booking_seat (booking_id, seat_id, price)
    SELECT o_booking_id, s.seat_id, sp.price
    FROM seat s
        JOIN show_price sp ON sp.show_id = p_show_id AND sp.seat_type_id = s.seat_type_id
    WHERE s.seat_id = v_seat_id;

    IF ROW_COUNT() <> 1 THEN
        -- no price configured for this seat type
        CLOSE seat_cursor;
        ROLLBACK;
        SET o_booking_id = NULL;
        SET o_status = 'INVALID_SEATS';
        LEAVE proc;
    END IF;

    UPDATE show_seat
    SET booking_id = o_booking_id,
        version = version + 1
    WHERE show_id = p_show_id
        AND seat_id = v_seat_id;
END LOOP;
CLOSE seat_cursor;

COMMIT;
SET o_status = 'HELD';

```

END\$\$

-- -----
-- *sp_create_payment: open a gateway order for a live hold. The amount is*
-- *computed from booking_seat (never trusted from the client).*
-- -----

```
CREATE PROCEDURE sp_create_payment(  
    IN p_booking_id      BIGINT UNSIGNED,  
    IN p_gateway         VARCHAR(20),  
    IN p_gateway_order_id VARCHAR(64),  
    OUT o_payment_id     BIGINT UNSIGNED,  
    OUT o_amount         DECIMAL(10, 2))  
BEGIN  
    SET o_payment_id = NULL;  
    SET o_amount = NULL;  
  
    IF EXISTS (SELECT 1 FROM booking  
               WHERE booking_id = p_booking_id AND status = 'PENDING' AND hold_expires_at > NOW(3)) THEN  
        INSERT INTO payment (booking_id, gateway, gateway_order_id, amount)  
        SELECT booking_id, p_gateway, p_gateway_order_id, SUM(price)  
        FROM booking_seat  
        WHERE booking_id = p_booking_id  
        GROUP BY booking_id;  
        SET o_payment_id = LAST_INSERT_ID();  
        SET o_amount = (SELECT amount FROM payment WHERE payment_id = o_payment_id);  
    END IF;  
END$$
```

-- -----
-- *sp_process_payment_webhook: idempotent handler for gateway callbacks.*
-- ** The same event delivered N times (even concurrently) is applied once:*
-- *the PRIMARY KEY (gateway, event_id) admits one insert; the others hit*
-- *duplicate-key and return DUPLICATE_EVENT.*
-- ** A second, different event for an already-captured payment is ignored*
-- *(state machine: only CREATED/FAILED -> CAPTURED).*
-- ** A payment that arrives after the hold lapsed confirms ONLY if nobody has*
-- *taken any of the seats meanwhile; otherwise it is flagged for refund.*
-- *o_result: BOOKING_CONFIRMED | DUPLICATE_EVENT | ALREADY_APPLIED |*
-- *LATE_PAYMENT_REFUND | DUPLICATE_PAYMENT_REFUND | AMOUNT_MISMATCH |*
-- *PAYMENT_FAILED | UNKNOWN_ORDER | IGNORED_EVENT*
-- -----

```
CREATE PROCEDURE sp_process_payment_webhook(  
    IN p_gateway         VARCHAR(20),  
    IN p_event_id        VARCHAR(64),  
    IN p_event_type      VARCHAR(40),  
    IN p_gateway_order_id VARCHAR(64),  
    IN p_gateway_payment_id VARCHAR(64),  
    IN p_amount          DECIMAL(10, 2),  
    IN p_payload          JSON,  
    OUT o_result          VARCHAR(40))  
proc:  
BEGIN  
    DECLARE v_duplicate TINYINT DEFAULT 0;  
    DECLARE v_found     TINYINT DEFAULT 0;  
    DECLARE v_payment_id BIGINT UNSIGNED;  
    DECLARE v_booking_id BIGINT UNSIGNED;  
    DECLARE v_show_id    BIGINT UNSIGNED;  
    DECLARE v_pay_status VARCHAR(20);  
    DECLARE v_pay_amount DECIMAL(10, 2);  
    DECLARE v_booking_status VARCHAR(20);  
    DECLARE v_total       INT;  
    DECLARE v_owned       INT;  
  
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_found = 0;  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;          -- event row rolled back too, so a gateway retry is processed cleanly  
        RESIGNAL;  
    END;  
END
```

```

END;

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;    -- see sp_hold_seats
START TRANSACTION;

-- 1) Dedupe: record the event; a duplicate key means it was already handled
BEGIN
    DECLARE CONTINUE HANDLER FOR 1062 SET v_duplicate = 1;
    INSERT INTO payment_webhook_event (gateway, event_id, event_type, payload)
    VALUES (p_gateway, p_event_id, p_event_type, p_payload);
END;
IF v_duplicate = 1 THEN
    ROLLBACK;
    SET o_result = 'DUPLICATE_EVENT';
    LEAVE proc;
END IF;

-- 2) Lock the payment row (serialises different events of the same payment)
SET v_found = 1;
SELECT payment_id, booking_id, status, amount
INTO v_payment_id, v_booking_id, v_pay_status, v_pay_amount
FROM payment
WHERE gateway = p_gateway AND gateway_order_id = p_gateway_order_id
FOR UPDATE;

IF v_found = 0 THEN
    SET o_result = 'UNKNOWN_ORDER';
ELSEIF p_event_type = 'payment.failed' THEN
    IF v_pay_status = 'CREATED' THEN
        UPDATE payment SET status = 'FAILED', gateway_payment_id = p_gateway_payment_id
        WHERE payment_id = v_payment_id;
        SET o_result = 'PAYMENT_FAILED';    -- hold stays; user may retry until it expires
    ELSE
        SET o_result = 'ALREADY_APPLIED';
    END IF;
ELSEIF p_event_type <> 'payment.captured' THEN
    SET o_result = 'IGNORED_EVENT';
ELSEIF v_pay_status NOT IN ('CREATED', 'FAILED') THEN
    SET o_result = 'ALREADY_APPLIED';
ELSEIF p_amount <> v_pay_amount THEN
    UPDATE payment SET status = 'REFUND_PENDING', gateway_payment_id = p_gateway_payment_id
    WHERE payment_id = v_payment_id;
    SET o_result = 'AMOUNT_MISMATCH';
ELSE
    -- 3) Lock the booking's seats FIRST (same order as sp_hold_seats), then the booking
    -- plain SELECT ... INTO = non-locking read. (SET v = (SELECT ...) would take a
    -- shared lock on the booking row BEFORE the seats - breaking the lock order.)
    SELECT show_id INTO v_show_id FROM booking WHERE booking_id = v_booking_id;

    SELECT COUNT(*), COALESCE(SUM(ss.booking_id <=> v_booking_id), 0)
    INTO v_total, v_owned
    FROM booking_seat bs
        JOIN show_seat ss ON ss.show_id = v_show_id AND ss.seat_id = bs.seat_id
    WHERE bs.booking_id = v_booking_id
    FOR UPDATE OF ss;

    SELECT status INTO v_booking_status
    FROM booking WHERE booking_id = v_booking_id
    FOR UPDATE;

    IF v_booking_status = 'CONFIRMED' THEN
        -- booking already paid through another order: refund this one, keep seats
        UPDATE payment SET status = 'REFUND_PENDING', gateway_payment_id = p_gateway_payment_id
        WHERE payment_id = v_payment_id;
        SET o_result = 'DUPLICATE_PAYMENT_REFUND';
    ELSEIF v_booking_status = 'PENDING' AND v_total > 0 AND v_owned = v_total THEN
        -- still owns every seat (even if the timer just lapsed, nobody took them)
        UPDATE booking SET status = 'CONFIRMED', confirmed_at = NOW(3)

```

```

        WHERE booking_id = v_booking_id;
        UPDATE payment SET status = 'CAPTURED', gateway_payment_id = p_gateway_payment_id
        WHERE payment_id = v_payment_id;
        SET o_result = 'BOOKING_CONFIRMED';
    ELSE
        -- hold lost (some seats re-sold): release what is left, refund the money
        UPDATE show_seat SET booking_id = NULL, version = version + 1
        WHERE show_id = v_show_id AND booking_id = v_booking_id;
        UPDATE booking SET status = 'EXPIRED'
        WHERE booking_id = v_booking_id AND status = 'PENDING';
        UPDATE payment SET status = 'REFUND_PENDING', gateway_payment_id = p_gateway_payment_id
        WHERE payment_id = v_payment_id;
        SET o_result = 'LATE_PAYMENT_REFUND';
    END IF;
END IF;

UPDATE payment_webhook_event
SET payment_id = v_payment_id, outcome = o_result, processed_at = NOW(3)
WHERE gateway = p_gateway AND event_id = p_event_id;

COMMIT;
END$$

-- -----
-- sp_release_expired_holds: housekeeping job (run every ~30 s by a scheduler or
-- the MySQL EVENT below). Correctness does NOT depend on it - expired holds are
-- already bookable (lazy expiry) - it just returns seats to NULL and marks
-- bookings EXPIRED so dashboards and seat maps stay tidy. One booking per
-- transaction keeps each lock set tiny; lock order is seats -> booking.
-- -----

CREATE PROCEDURE sp_release_expired_holds(IN p_batch_size INT, OUT o_released INT)
BEGIN
    DECLARE v_booking_id BIGINT UNSIGNED;
    DECLARE v_done        TINYINT DEFAULT 0;
    DECLARE v_status      VARCHAR(20);
    DECLARE v_expiry      DATETIME(3);
    DECLARE v_dummy       INT;

    DECLARE expired_cursor CURSOR FOR
        SELECT booking_id FROM booking
        WHERE status = 'PENDING' AND hold_expires_at <= NOW(3)
        ORDER BY hold_expires_at
        LIMIT p_batch_size;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    SET o_released = 0;
    OPEN expired_cursor;
    release_loop:
    LOOP
        FETCH expired_cursor INTO v_booking_id;
        IF v_done = 1 THEN
            LEAVE release_loop;
        END IF;

        SET TRANSACTION ISOLATION LEVEL READ COMMITTED; -- see sp_hold_seats
        START TRANSACTION;
        -- seats first ...
        SELECT COUNT(*) INTO v_dummy
        FROM show_seat
        WHERE booking_id = v_booking_id
        FOR UPDATE;
        -- ... then the booking, re-checked under lock (a webhook may have confirmed it)
        SELECT status, hold_expires_at INTO v_status, v_expiry

```

```

FROM booking WHERE booking_id = v_booking_id
FOR UPDATE;

IF v_status = 'PENDING' AND v_expiry <= NOW(3) THEN
    UPDATE show_seat SET booking_id = NULL, version = version + 1
    WHERE booking_id = v_booking_id;
    UPDATE booking SET status = 'EXPIRED' WHERE booking_id = v_booking_id;
    SET o_released = o_released + 1;
END IF;
COMMIT;
END LOOP;
CLOSE expired_cursor;
END$$

DELIMITER ;

-- Optional: Let MySQL run the clean-up itself (requires event_scheduler=ON).
DROP EVENT IF EXISTS ev_release_expired_holds;
CREATE EVENT ev_release_expired_holds
ON SCHEDULE EVERY 30 SECOND
DO CALL sp_release_expired_holds(500, @released);

```

Appendix C – P1 SQL: sample data (sql/03_sample_data.sql)

```
-- =====
-- BookMyShow-style Ticketing Backend | P1 - Sample data
-- Run AFTER 01_schema.sql and 02_procedures.sql:
--     mysql -u root -p bookmyshow < sql/03_sample_data.sql
--
-- Show dates are generated RELATIVE TO TODAY (CURDATE() .. CURDATE()+6), exactly
-- Like the app's "next 7 dates" strip, so P2 returns rows whenever you run it.
-- Theatre, movie and customer names are fictional.
-- =====

USE bookmyshow;

-- ----- Locations
INSERT INTO city (city_id, name, state) VALUES
    (1, 'Hyderabad', 'Telangana'),
    (2, 'Bengaluru', 'Karnataka');

INSERT INTO theatre (theatre_id, city_id, name, address_line) VALUES
    (1, 1, 'Aurora Cinemas: Kukatpally', '4th Floor, Metro Square Mall, Kukatpally, Hyderabad 500072'),
    (2, 1, 'Starlight Multiplex: Gachibowli', 'Level 3, Orbit Towers, Gachibowli, Hyderabad 500032'),
    (3, 2, 'Galaxy Screens: Koramangala', '80 Feet Road, 4th Block, Koramangala, Bengaluru 560034');

INSERT INTO amenity (amenity_id, name) VALUES
    (1, 'M-Ticket'), (2, 'Food & Beverage'), (3, 'Parking'), (4, 'Wheelchair Access');

INSERT INTO theatre_amenity (theatre_id, amenity_id) VALUES
    (1, 1), (1, 2), (1, 3), (1, 4),
    (2, 1), (2, 2),
    (3, 1), (3, 2), (3, 3);

INSERT INTO screen (screen_id, theatre_id, name) VALUES
    (1, 1, 'Audi 1'), (2, 1, 'Audi 2'), (3, 1, 'IMAX'),
    (4, 2, 'Screen 1'),
    (5, 3, 'Screen 1');

INSERT INTO seat_type (seat_type_id, name) VALUES
    (1, 'CLASSIC'), (2, 'PRIME'), (3, 'RECLINER');

-- 80 seats per screen: rows A-H x seats 1-10.
-- A-D = CLASSIC, E-G = PRIME, H = RECLINER (back row).
INSERT INTO seat (screen_id, row_label, seat_number, seat_type_id)
SELECT sc.screen_id,
       r.row_label,
       n.seat_number,
       CASE WHEN r.row_label IN ('A', 'B', 'C', 'D') THEN 1
            WHEN r.row_label IN ('E', 'F', 'G') THEN 2
            ELSE 3 END
FROM screen sc
CROSS JOIN (SELECT 'A' AS row_label UNION ALL SELECT 'B' UNION ALL SELECT 'C' UNION ALL SELECT 'D'
             UNION ALL SELECT 'E' UNION ALL SELECT 'F' UNION ALL SELECT 'G' UNION ALL SELECT 'H') AS r
CROSS JOIN (SELECT 1 AS seat_number UNION ALL SELECT 2 UNION ALL SELECT 3 UNION ALL SELECT 4
             UNION ALL SELECT 5 UNION ALL SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8
             UNION ALL SELECT 9 UNION ALL SELECT 10) AS n
ORDER BY sc.screen_id, r.row_label, n.seat_number;

-- ----- catalogue
INSERT INTO movie (movie_id, title, duration_min, certificate, release_date) VALUES
    (1, 'Monsoon Heist', 152, 'UA', '2026-09-18'),
    (2, 'Iron Valley', 169, 'UA', '2026-09-25'),
    (3, 'Chasing Tides', 134, 'U', '2026-09-11'),
    (4, 'Little Astronaut', 108, 'U', '2026-08-28'),
    (5, 'The Midnight Ledger', 141, 'A', '2026-09-25');

INSERT INTO genre (genre_id, name) VALUES
```

```

(1, 'Action'), (2, 'Thriller'), (3, 'Drama'), (4, 'Family'),
(5, 'Animation'), (6, 'Sci-Fi'), (7, 'Crime');

INSERT INTO movie_genre (movie_id, genre_id) VALUES
(1, 1), (1, 2), (1, 7),
(2, 1), (2, 3),
(3, 3), (3, 4),
(4, 4), (4, 5), (4, 6),
(5, 2), (5, 7);

INSERT INTO language (language_id, name) VALUES
(1, 'English'), (2, 'Hindi'), (3, 'Telugu'), (4, 'Tamil'), (5, 'Kannada');

INSERT INTO show_format (format_id, name) VALUES
(1, '2D'), (2, '3D'), (3, 'IMAX 2D'), (4, '4DX');

-- ----- shows
-- A daily schedule template per screen, repeated for today + next 6 days.
-- Start times are spaced so shows on a screen never overlap (film + cleaning).
INSERT INTO shows (movie_id, screen_id, language_id, format_id, start_time)
SELECT t.movie_id, t.screen_id, t.language_id, t.format_id,
TIMESTAMP(CURDATE() + INTERVAL d.day_offset DAY, t.show_time)
FROM (SELECT 0 AS day_offset UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL SELECT 3
UNION ALL SELECT 4 UNION ALL SELECT 5 UNION ALL SELECT 6) AS d
CROSS JOIN (
-- movie screen lang fmt time
SELECT 2 AS movie_id, 1 AS screen_id, 3 AS language_id, 1 AS format_id, '10:00:00' AS show_time
UNION ALL SELECT 2, 1, 3, 1, '13:45:00'
UNION ALL SELECT 1, 1, 2, 1, '17:30:00'
UNION ALL SELECT 2, 1, 3, 1, '21:15:00'
UNION ALL SELECT 4, 2, 1, 1, '09:30:00'
UNION ALL SELECT 3, 2, 3, 1, '12:00:00'
UNION ALL SELECT 1, 2, 3, 1, '15:00:00'
UNION ALL SELECT 5, 2, 1, 1, '18:30:00'
UNION ALL SELECT 3, 2, 2, 1, '22:00:00'
UNION ALL SELECT 2, 3, 3, 3, '11:00:00'
UNION ALL SELECT 2, 3, 2, 3, '14:30:00'
UNION ALL SELECT 2, 3, 3, 3, '18:15:00'
UNION ALL SELECT 5, 3, 1, 3, '22:00:00'
UNION ALL SELECT 1, 4, 2, 1, '10:30:00'
UNION ALL SELECT 3, 4, 3, 2, '14:00:00'
UNION ALL SELECT 2, 4, 3, 1, '18:00:00'
UNION ALL SELECT 5, 4, 1, 1, '21:30:00'
UNION ALL SELECT 4, 5, 5, 1, '11:00:00'
UNION ALL SELECT 1, 5, 2, 1, '15:00:00'
UNION ALL SELECT 2, 5, 3, 1, '19:00:00'
UNION ALL SELECT 5, 5, 1, 1, '22:15:00'
) AS t
ORDER BY d.day_offset, t.screen_id, t.show_time;

-- One show cancelled by the theatre (P2 must not list it)
UPDATE shows
SET status = 'CANCELLED'
WHERE screen_id = 2
AND start_time = TIMESTAMP(CURDATE() + INTERVAL 1 DAY, '22:00:00');

-- Prices per (show, seat type): CLASSIC 180 / PRIME 250 / RECLINER 450, +150 for IMAX
INSERT INTO show_price (show_id, seat_type_id, price)
SELECT s.show_id,
st.seat_type_id,
CASE st.name WHEN 'CLASSIC' THEN 180 WHEN 'PRIME' THEN 250 ELSE 450 END
+ IF(f.name = 'IMAX 2D', 150, 0)
FROM shows s
JOIN show_format f ON f.format_id = s.format_id
CROSS JOIN seat_type st;

-- Seat inventory: one row per (show, seat) - created when the show is scheduled
INSERT INTO show_seat (show_id, seat_id)

```

```

SELECT s.show_id, se.seat_id
FROM shows s
      JOIN seat se ON se.screen_id = s.screen_id;

----- customers
INSERT INTO users (user_id, full_name, email, phone) VALUES
  (1, 'Ananya Rao', 'ananya.rao@example.com', '9800000001'),
  (2, 'Rahul Verma', 'rahul.verma@example.com', '9800000002'),
  (3, 'Priya Nair', 'priya.nair@example.com', '9800000003'),
  (4, 'Karthik Reddy', 'karthik.reddy@example.com', '9800000004'),
  (5, 'Meera Iyer', 'meera.iyer@example.com', '9800000005');

----- bookings
-- All three sample bookings are for tomorrow 21:15, Audi 1, Aurora Cinemas.
SET @demo_show = (SELECT show_id FROM shows
                  WHERE screen_id = 1 AND start_time = TIMESTAMP(CURDATE() + INTERVAL 1 DAY, '21:15:00'));

-- Booking 1: CONFIRMED (paid) - recliners H5, H6
-- Booking 2: PENDING hold, expires in 10 minutes - PRIME E5, E6, E7
-- Booking 3: EXPIRED hold (user never paid) - CLASSIC A1; seat already released
INSERT INTO booking (booking_id, user_id, show_id, status, idempotency_key, hold_expires_at, created_at,
confirmed_at) VALUES
  (1, 1, @demo_show, 'CONFIRMED', '6f1c1b4e-2a51-4a4e-9a0e-1f6d9a1b0001',
NOW(3) - INTERVAL 50 MINUTE, NOW(3) - INTERVAL 60 MINUTE, NOW(3) - INTERVAL 57 MINUTE),
  (2, 2, @demo_show, 'PENDING', '6f1c1b4e-2a51-4a4e-9a0e-1f6d9a1b0002',
NOW(3) + INTERVAL 10 MINUTE, NOW(3), NULL),
  (3, 3, @demo_show, 'EXPIRED', '6f1c1b4e-2a51-4a4e-9a0e-1f6d9a1b0003',
NOW(3) - INTERVAL 20 MINUTE, NOW(3) - INTERVAL 30 MINUTE, NULL);

INSERT INTO booking_seat (booking_id, seat_id, price)
SELECT b.booking_id, se.seat_id, sp.price
FROM (SELECT 1 AS booking_id, 'H' AS row_label, 5 AS seat_number
      UNION ALL SELECT 1, 'H', 6
      UNION ALL SELECT 2, 'E', 5
      UNION ALL SELECT 2, 'E', 6
      UNION ALL SELECT 2, 'E', 7
      UNION ALL SELECT 3, 'A', 1) AS b
      JOIN seat se ON se.screen_id = 1 AND se.row_label = b.row_label AND se.seat_number = b.seat_number
      JOIN show_price sp ON sp.show_id = @demo_show AND sp.seat_type_id = se.seat_type_id;

-- Current owners in the inventory (booking 3 expired, so A1 stays NULL / free)
UPDATE show_seat ss
      JOIN booking_seat bs ON bs.seat_id = ss.seat_id
SET ss.booking_id = bs.booking_id,
    ss.version = ss.version + 1
WHERE ss.show_id = @demo_show
      AND bs.booking_id IN (1, 2);

----- payments
INSERT INTO payment (payment_id, booking_id, gateway, gateway_order_id, gateway_payment_id, amount, status,
created_at) VALUES
  (1, 1, 'RAZORPAY', 'order_Q1A2B3C4D5', 'pay_Q9Z8Y7X6W5', 900.00, 'CAPTURED', NOW(3) - INTERVAL 59 MINUTE),
  (2, 2, 'RAZORPAY', 'order_Q1A2B3C4D6', NULL, 750.00, 'CREATED', NOW(3));

INSERT INTO payment_webhook_event (gateway, event_id, event_type, payment_id, payload, outcome, received_at,
processed_at) VALUES
  ('RAZORPAY', 'evt_Qa1b2c3d4e5', 'payment.captured', 1,
JSON_OBJECT('order_id', 'order_Q1A2B3C4D5', 'payment_id', 'pay_Q9Z8Y7X6W5', 'amount', 900.00, 'currency',
'INR')),
  ('BOOKING_CONFIRMED', NOW(3) - INTERVAL 57 MINUTE, NOW(3) - INTERVAL 57 MINUTE);

```

Appendix D – P2 SQL

(sql/04_p2_shows_by_theatre_and_date.sql)

```
-- =====
-- BookMyShow-style Ticketing Backend | P2
-- "List all the shows on a given date at a given theatre, with their timings."
-- Run: mysql -u root -p bookmyshow < sql/04_p2_shows_by_theatre_and_date.sql
--
-- Inputs (change these two lines):
--     @theatre_id - the theatre the user opened
--     @show_date   - the date chip the user tapped in the 7-day strip
-- =====

USE bookmyshow;

SET @theatre_id = 1;
SET @show_date  = CURDATE() + INTERVAL 1 DAY;    -- e.g. DATE('2026-09-29')

-- -----
-- P2 (main answer): one row per show, ordered by movie then time.
--
-- Why "start_time >= @d AND start_time < @d + 1 day" and not DATE(start_time) = @d?
-- Wrapping the column in a function hides it from the index. The half-open
-- range is SARGable: MySQL walks screen(theatre_id) -> shows(screen_id,
-- start_time) with an index range scan and touches only that day's rows.
-- -----

SELECT t.name                AS theatre,
       m.title               AS movie,
       m.certificate,
       l.name                AS language,
       f.name                AS format,
       sc.name               AS screen,
       s.show_id,
       TIME_FORMAT(s.start_time, '%h:%i %p') AS show_time,
       s.cancellation_allowed AS cancellable
FROM theatre t
      JOIN screen sc    ON sc.theatre_id = t.theatre_id
      JOIN shows s      ON s.screen_id   = sc.screen_id
      JOIN movie m      ON m.movie_id    = s.movie_id
      JOIN language l   ON l.language_id = s.language_id
      JOIN show_format f ON f.format_id   = s.format_id
WHERE t.theatre_id = @theatre_id
      AND s.start_time >= @show_date
      AND s.start_time < @show_date + INTERVAL 1 DAY
      AND s.status = 'SCHEDULED'
ORDER BY m.title, l.name, f.name, s.start_time;

-- -----
-- P2 (UI shape): exactly what the theatre page renders - one card per
-- movie + language + format with its show timings side by side.
-- -----

SELECT m.title                AS movie,
       m.certificate,
       l.name                AS language,
       f.name                AS format,
       COUNT(*)              AS shows,
       GROUP_CONCAT(TIME_FORMAT(s.start_time, '%h:%i %p')
                    ORDER BY s.start_time SEPARATOR ' | ') AS show_timings
FROM screen sc
      JOIN shows s      ON s.screen_id   = sc.screen_id
      JOIN movie m      ON m.movie_id    = s.movie_id
      JOIN language l   ON l.language_id = s.language_id
      JOIN show_format f ON f.format_id   = s.format_id
WHERE sc.theatre_id = @theatre_id
```

```

AND s.start_time >= @show_date
AND s.start_time < @show_date + INTERVAL 1 DAY
AND s.status = 'SCHEDULED'
GROUP BY m.movie_id, m.title, m.certificate, l.name, f.name
ORDER BY m.title, l.name, f.name;

-- -----
-- Supporting query: the 7-day date strip at the top of the theatre page
-- (only dates that actually have at least one bookable show).
-- -----

SELECT DATE(s.start_time) AS show_date,
       DATE_FORMAT(s.start_time, '%a %d %b') AS label,
       COUNT(*) AS shows
FROM screen sc
      JOIN shows s ON s.screen_id = sc.screen_id
WHERE sc.theatre_id = @theatre_id
      AND s.start_time >= CURDATE()
      AND s.start_time < CURDATE() + INTERVAL 7 DAY
      AND s.status = 'SCHEDULED'
GROUP BY DATE(s.start_time), DATE_FORMAT(s.start_time, '%a %d %b')
ORDER BY show_date;

-- -----
-- Proof that P2 is index-driven (no full table scan on shows).
-- -----

EXPLAIN
SELECT m.title, s.start_time
FROM screen sc
      JOIN shows s ON s.screen_id = sc.screen_id
      JOIN movie m ON m.movie_id = s.movie_id
WHERE sc.theatre_id = @theatre_id
      AND s.start_time >= @show_date
      AND s.start_time < @show_date + INTERVAL 1 DAY
      AND s.status = 'SCHEDULED';

```

Appendix E – Walk-through output (sql/05_demo_and_checks.sql)

Hold → retry → conflict → payment → duplicate webhooks → timed expiry → late payment → clean-up, followed by the seat map and the integrity checks (every violation count must be 0).

```
+-----+-----+-----+
| step                | booking_id | status |
+-----+-----+-----+
| 1. Karthik holds F5,F6 |          4 | HELD   |
+-----+-----+-----+

+-----+-----+-----+
| step                | booking_id | status   |
+-----+-----+-----+
| 2. Same request retried |          4 | ALREADY_HELD |
+-----+-----+-----+

+-----+-----+-----+
| step                | booking_id | status   |
+-----+-----+-----+
| 3. Meera tries F6 |         NULL | SEATS_UNAVAILABLE |
+-----+-----+-----+

+-----+-----+-----+
| step                | booking_id | status   |
+-----+-----+-----+
| 4. Meera tries E5 |         NULL | SEATS_UNAVAILABLE |
+-----+-----+-----+

+-----+-----+-----+
| step                | payment_id | amount |
+-----+-----+-----+
| 5a. Payment order created |          3 | 500.00 |
+-----+-----+-----+

+-----+-----+-----+
| step                | outcome    |
+-----+-----+-----+
| 5b. Webhook delivery #1 | BOOKING_CONFIRMED |
+-----+-----+-----+

+-----+-----+-----+
| step                | outcome    |
+-----+-----+-----+
| 5c. Webhook delivery #2 (duplicate) | DUPLICATE_EVENT |
+-----+-----+-----+

+-----+-----+-----+
| step                | outcome    |
+-----+-----+-----+
| 5d. Different event id, same payment | ALREADY_APPLIED |
+-----+-----+-----+

+-----+-----+-----+
| step                | booking_id | status |
+-----+-----+-----+
| 6a. Priya holds G1 for 1 s |          7 | HELD   |
+-----+-----+-----+

+-----+-----+-----+
| step                | booking_id | status |
+-----+-----+-----+
| 7. Rahul takes G1 after expiry |          8 | HELD   |
+-----+-----+-----+

+-----+-----+-----+
| step                | outcome    |
+-----+-----+-----+
| 8. Priya's late payment | LATE_PAYMENT_REFUND |
+-----+-----+-----+

+-----+-----+-----+
| step                | released   |
+-----+-----+-----+
| 9. Expired holds released by job |          0 |
+-----+-----+-----+

+-----+-----+-----+
| row_label | seats |
+-----+-----+-----+
```

A	1:FREE 2:FREE 3:FREE 4:FREE 5:FREE 6:FREE 7:FREE 8:FREE 9:FREE 10:FREE
B	1:FREE 2:FREE 3:FREE 4:FREE 5:FREE 6:FREE 7:FREE 8:FREE 9:FREE 10:FREE
C	1:FREE 2:FREE 3:FREE 4:FREE 5:FREE 6:FREE 7:FREE 8:FREE 9:FREE 10:FREE
D	1:FREE 2:FREE 3:FREE 4:FREE 5:FREE 6:FREE 7:FREE 8:FREE 9:FREE 10:FREE
E	1:FREE 2:FREE 3:FREE 4:FREE 5:HELD 6:HELD 7:HELD 8:FREE 9:FREE 10:FREE
F	1:FREE 2:FREE 3:FREE 4:FREE 5:BOOKED 6:BOOKED 7:FREE 8:FREE 9:FREE 10:FREE
G	1:HELD 2:FREE 3:FREE 4:FREE 5:FREE 6:FREE 7:FREE 8:FREE 9:FREE 10:FREE
H	1:FREE 2:FREE 3:FREE 4:FREE 5:BOOKED 6:BOOKED 7:FREE 8:FREE 9:FREE 10:FREE

step	rows_won
Optimistic CAS on H1	1

invariant	violations
A seat in two CONFIRMED bookings of the same show	0
CONFIRMED booking not owning all its seats	0
Live hold not owning all its seats (lost hold)	0
CAPTURED payment whose booking is not CONFIRMED	0
Booking with more than one CAPTURED payment	0
Webhook event received but never processed	0